

The toral subgroup of $G(\mathfrak{m})$ normalizes the unipotent subgroup: a machine-checked proof

Abstract

We present a formal verification of Theorem 3.4 of the source. The theorem concerns the group $G(\mathfrak{m})$, defined by generators and relations and associated with the Monster Lie algebra $\mathfrak{m}$. It states that the toral subgroup H , generated by the elements $H_1(s)$ and $H_2(s)$, normalizes the unipotent subgroup U , generated by all unipotent generators. The formalization replaces the coefficients $c(j)$ of the modular function J by an arbitrary function $c: \mathbb{N} \rightarrow \mathbb{N}$, and the theorem is proved with no hypothesis on c . The verified proof follows the source argument: conjugation by a generator of H sends a unipotent generator to another unipotent generator, and this extends to the generated subgroups. The one point the source leaves implicit is checked explicitly, namely that the inverses of the generators of H are again generators of H .

Source	Carbone et al., G-Simple (draft), GSimple_Theorem_3_4
Library	libraries/gsimple.lean, 45 declarations
Lean	leanprover/lean4:v4.23.0, Mathlib 37df177aaa
Machine checks	compiles; no sorry; statement matches target; module build
Axioms	propext, Classical.choice, Quot.sound
Independent review	FAITHFUL (claude-opus-5-5)
Agents	captain claude-opus-5-5; solvers aiprover; report claude-opus-5-5
Run	srv_20261006_225233_GSimple_Theorem_3_4

Contents

1	Statement	1
2	Formalization	4
2.1	The library	4
2.2	The subgroups H and U	8
2.3	The target statement	9
3	Proof	9
3.1	Overview	9
3.2	Proof of the theorem	16
4	Independent review	17
A	Complete Lean file	18

1 Statement

Let $\mathbb{C}$ denote the complex numbers and $\mathbb{N}$ the natural numbers. Let $c(j)$ be the coefficient of q^j in the modular function

$$J(q) = \sum_{j \geq -1} c(j)q^j = \frac{1}{q} + 196884q + 21493760q^2 + 864299970q^3 + \dots,$$

so that $c(-1) = 1$, $c(0) = 0$, $c(1) = 196884, \dots$.

Index set and constants. Define the index set

$$E = \{(\ell, j, k) \mid j \in \mathbb{N}, 1 \leq k \leq c(j), 0 \leq \ell < j\}.$$

For indices ℓ, j put

$$c_{\ell j} := (-1)^{\ell+1} \binom{j-1}{\ell} (\ell+1)(j-\ell) \in \mathbb{C}.$$

Generators. Let $\mathcal{X}$ be the set of symbols

$$\mathcal{X} = \{H_1(s), H_2(s), X_{-1}(u), Y_{-1}(u), X_{\ell,jk}(u), Y_{\ell,jk}(u) \mid s \in \mathbb{C}^\times, u \in \mathbb{C}, (\ell, j, k) \in E\},$$

and let $F(\mathcal{X})$ be the free group on $\mathcal{X}$. The symbols $X_{-1}(u), Y_{-1}(u), X_{\ell,jk}(u), Y_{\ell,jk}(u)$ are called *unipotent generators*. The commutator is $(a, b) := aba^{-1}b^{-1}$.

Relations. For all $s, t \in \mathbb{C}^\times$, $u, v \in \mathbb{C}$ and $(\ell, j, k), (m, p, q) \in E$, the set $\mathcal{R} \subseteq F(\mathcal{X})$ and the auxiliary elements $\tilde{w}$ are given as follows.

$$\tilde{w}_{-1}(s) := X_{-1}(s)Y_{-1}(-s^{-1})X_{-1}(s), \quad \tilde{w}_{-1} := \tilde{w}_{-1}(1), \quad (\text{Re:0})$$

$$\tilde{w}_{\ell,jk}(s) := X_{\ell,jk}(s)Y_{\ell,jk}\left(\frac{-s^{-1}}{c_{\ell j}}\right)X_{\ell,jk}(s), \quad \tilde{w}_{\ell,jk} := \tilde{w}_{\ell,jk}(1), \quad (\text{Im:0})$$

relations defining $\text{GL}_2(-1)$:

$$H_1(s)H_1(t) = H_1(st), \quad (\text{H:1a})$$

$$H_2(s)H_2(t) = H_2(st), \quad (\text{H:1b})$$

$$H_1(s)H_2(t) = H_2(t)H_1(s), \quad (\text{H:2})$$

$$X_{-1}(u)X_{-1}(v) = X_{-1}(u+v), \quad (\text{Re:1a})$$

$$Y_{-1}(u)Y_{-1}(v) = Y_{-1}(u+v), \quad (\text{Re:1b})$$

$$Y_{-1}(-t)X_{-1}(s)Y_{-1}(t) = X_{-1}(-t^{-1})Y_{-1}(-t^2s)X_{-1}(t^{-1}), \quad (\text{Re:2})$$

$$\tilde{w}_{-1}(s)\tilde{w}_{-1} = H_1(-s)H_2(-s^{-1}), \quad (\text{Re:3})$$

$$\tilde{w}_{-1}X_{-1}(u)\tilde{w}_{-1}^{-1} = Y_{-1}(-u), \quad (\text{Re:4a})$$

$$\tilde{w}_{-1}Y_{-1}(u)\tilde{w}_{-1}^{-1} = X_{-1}(-u), \quad (\text{Re:4b})$$

$$\tilde{w}_{-1}H_1(s)\tilde{w}_{-1}^{-1} = H_2(s), \quad (\text{Re:5a})$$

$$\tilde{w}_{-1}H_2(s)\tilde{w}_{-1}^{-1} = H_1(s), \quad (\text{Re:5b})$$

$$H_1(s)X_{-1}(u)H_1(s)^{-1} = X_{-1}(su), \quad (\text{Re:6a})$$

$$H_2(s)X_{-1}(u)H_2(s)^{-1} = X_{-1}(s^{-1}u), \quad (\text{Re:6b})$$

$$H_1(s)Y_{-1}(u)H_1(s)^{-1} = Y_{-1}(s^{-1}u), \quad (\text{Re:6c})$$

$$H_2(s)Y_{-1}(u)H_2(s)^{-1} = Y_{-1}(su), \quad (\text{Re:6d})$$

relations defining $\mathrm{GL}_2(\ell, j, k)$:

$$X_{\ell,jk}(u)X_{\ell,jk}(v) = X_{\ell,jk}(u+v), \quad (\text{Im:1a})$$

$$Y_{\ell,jk}(u)Y_{\ell,jk}(v) = Y_{\ell,jk}(u+v), \quad (\text{Im:1b})$$

$$Y_{\ell,jk}(-t)X_{\ell,jk}(s)Y_{\ell,jk}(t) = X_{\ell,jk}\left(\frac{-t^{-1}}{c_{\ell j}}\right)Y_{\ell,jk}(-c_{\ell j}t^2s)X_{\ell,jk}\left(\frac{t^{-1}}{c_{\ell j}}\right), \quad (\text{Im:2})$$

$$\tilde{w}_{\ell,jk}(s^{(\ell+1)(j-\ell)})\tilde{w}_{\ell,jk} = H_1((-s)^{j-\ell})H_2((-s)^{\ell+1}), \quad (\text{Im:3})$$

$$\tilde{w}_{\ell,jk}X_{\ell,jk}(u)\tilde{w}_{\ell,jk}^{-1} = Y_{\ell,jk}\left(\frac{-u}{c_{\ell j}}\right), \quad (\text{Im:4a})$$

$$\tilde{w}_{\ell,jk}Y_{\ell,jk}(u)\tilde{w}_{\ell,jk}^{-1} = X_{\ell,jk}(-c_{\ell j}u), \quad (\text{Im:4b})$$

$$\tilde{w}_{\ell,jk}H_1(s^{j-\ell})\tilde{w}_{\ell,jk}^{-1} = H_2(s^{-(\ell+1)}), \quad (\text{Im:5a})$$

$$\tilde{w}_{\ell,jk}H_2(s^{\ell+1})\tilde{w}_{\ell,jk}^{-1} = H_1(s^{-(j-\ell)}), \quad (\text{Im:5b})$$

$$H_1(s)X_{\ell,jk}(u)H_1(s)^{-1} = X_{\ell,jk}(s^{\ell+1}u), \quad (\text{Im:6a})$$

$$H_2(s)X_{\ell,jk}(u)H_2(s)^{-1} = X_{\ell,jk}(s^{j-\ell}u), \quad (\text{Im:6b})$$

$$H_1(s)Y_{\ell,jk}(u)H_1(s)^{-1} = Y_{\ell,jk}(s^{-(\ell+1)}u), \quad (\text{Im:6c})$$

$$H_2(s)Y_{\ell,jk}(u)H_2(s)^{-1} = Y_{\ell,jk}(s^{-(j-\ell)}u), \quad (\text{Im:6d})$$

relations between unipotent generators:

$$(X_{-1}(u), X_{j-1,jk}(v)) = 1, \quad (\text{U:1a})$$

$$(Y_{-1}(u), Y_{j-1,jk}(v)) = 1, \quad (\text{U:1b})$$

$$(Y_{-1}(u), X_{0,jk}(v)) = 1, \quad (\text{U:1c})$$

$$(X_{-1}(u), Y_{0,jk}(v)) = 1, \quad (\text{U:1d})$$

$$(X_{\ell,jk}(u), Y_{m,pq}(v)) = 1 \quad \text{for } j \neq p, \text{ or } k \neq q, \text{ or } |\ell - m| > 1, \quad (\text{U:2})$$

$$(X_{1,2k}(u), Y_{0,2k}(v)) = X_{-1}(uv), \quad (\text{U:3a})$$

$$(X_{0,2k}(u), Y_{1,2k}(v)) = Y_{-1}(uv), \quad (\text{U:3b})$$

action of $\tilde{w}_{-1}$ on unipotent generators:

$$\tilde{w}_{-1}X_{\ell,jk}(u)\tilde{w}_{-1}^{-1} = X_{j-1-\ell,jk}((-1)^{j-\ell-1}u), \quad (\text{U:4a})$$

$$\tilde{w}_{-1}Y_{\ell,jk}(u)\tilde{w}_{-1}^{-1} = Y_{j-1-\ell,jk}((-1)^{j-\ell-1}u). \quad (\text{U:4b})$$

The group and its subgroups. The group is $G(\mathfrak{m}) = \langle \mathcal{X} \mid \mathcal{R} \rangle = F(\mathcal{X})/N_{\mathcal{R}}$, where $N_{\mathcal{R}}$ is the normal closure of $\mathcal{R}$.

- The *toral subgroup* $H \leq G(\mathfrak{m})$ is generated by $\{H_1(s), H_2(s) \mid s \in \mathbb{C}^\times\}$.
- The *unipotent subgroup* $U \leq G(\mathfrak{m})$ is generated by all unipotent generators:

$$U = \langle X_{-1}(u), Y_{-1}(u), X_{\ell,jk}(u), Y_{\ell,jk}(u) \mid u \in \mathbb{C}, (\ell, j, k) \in E \rangle.$$

Theorem 1.1 (Theorem 3.4 of the source). *The toral subgroup H normalizes the unipotent subgroup U . That is, for every $h \in H$ and every $u \in U$, the conjugate huh^{-1} lies in U .*

2 Formalization

2.1 The library

The presentation of $G(\mathbf{m})$ comes from an earlier verified development (the library). The same library also contains the proof that $G(\mathbf{m})$ is perfect (Theorem 3.3, `gsimple_g_m_perfect`) together with its auxiliary lemmas. Those results are not used here, apart from the two theorems named below.

Index set and constants.

- The coefficients of J are replaced by a parameter $c: \mathbb{N} \rightarrow \mathbb{N}$.
- The predicate `Valid c l j k` means $\ell < j$ and $1 \leq k \leq c(j)$, with $\ell, j, k \in \mathbb{N}$. This is exactly $(\ell, j, k) \in E$, with c in place of the coefficients of J .

Lean: `Valid` — library, verified earlier

```

| /-- Index set E:  $(\ell, j, k)$  with  $0 \leq \ell < j$  and  $1 \leq k \leq c(j)$ . -/
| def Valid (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) (l j k :  $\mathbb{N}$ ) : Prop := l < j  $\wedge$  1  $\leq$  k  $\wedge$  k  $\leq$  c j

```

Generators. The type `Gen c` is the set $\mathcal{X}$ of generating symbols:

- $H_1(s)$ and $H_2(s)$ with $s \in \mathbb{C}^\times$, a unit of $\mathbb{C}$;
- $X_{-1}(u)$ and $Y_{-1}(u)$ with $u \in \mathbb{C}$;
- $X_{\ell, jk}(u)$ and $Y_{\ell, jk}(u)$ with $u \in \mathbb{C}$ and a proof of `Valid c l j k`.

The membership proof is a proposition, so it carries no information. Its only role is to ensure that $X_{\ell, jk}$ and $Y_{\ell, jk}$ are indexed exactly by E .

Lean: `Gen` — library, verified earlier

```

| /-- The generating symbols. -/
| inductive Gen (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) : Type
| H1 :  $\mathbb{C}^\times \rightarrow$  Gen c
| H2 :  $\mathbb{C}^\times \rightarrow$  Gen c
| Xm :  $\mathbb{C} \rightarrow$  Gen c
| Ym :  $\mathbb{C} \rightarrow$  Gen c
| X (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u :  $\mathbb{C}$ ) : Gen c
| Y (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u :  $\mathbb{C}$ ) : Gen c

```

The free group $F(\mathcal{X})$ is `F`.

Lean: `F` — library, verified earlier

```

| abbrev F (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) := FreeGroup (Gen c)

```

Generators as words. `h1`, `h2`, `xm`, `ym`, `xx`, `yy` are the words of length one $H_1(s)$, $H_2(s)$, $X_{-1}(u)$, $Y_{-1}(u)$, $X_{\ell, jk}(u)$, $Y_{\ell, jk}(u)$ in $F(\mathcal{X})$.

Lean: h1 — library, verified earlier

```
| def h1 {c : ℕ → ℕ} (s : ℂ×) : F c := FreeGroup.of (Gen.H1 s)
```

Lean: h2 — library, verified earlier

```
| def h2 {c : ℕ → ℕ} (s : ℂ×) : F c := FreeGroup.of (Gen.H2 s)
```

Lean: xm — library, verified earlier

```
| def xm {c : ℕ → ℕ} (u : ℂ) : F c := FreeGroup.of (Gen.Xm u)
```

Lean: ym — library, verified earlier

```
| def ym {c : ℕ → ℕ} (u : ℂ) : F c := FreeGroup.of (Gen.Ym u)
```

Lean: xx — library, verified earlier

```
| def xx {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : F c :=  
  FreeGroup.of (Gen.X l j k h u)
```

Lean: yy — library, verified earlier

```
| def yy {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (u : ℂ) : F c :=  
  FreeGroup.of (Gen.Y l j k h u)
```

The constant $c_{\ell j}$. cc is the complex number $(-1)^{\ell+1} \binom{j-1}{\ell} (\ell+1)(j-\ell)$, where $j-1$ and $j-\ell$ are computed in $\mathbb{N}$. For $(\ell, j, k) \in E$ we have $\ell \leq j-1$, so all three factors are nonzero. Hence $c_{\ell j} \neq 0$, and the divisions by $c_{\ell j}$ in the relations mean what they mean in the source.

Lean: cc — library, verified earlier

```
| /- c_{\ell j} = (-1)^{(\ell+1)} * binom(j-1, \ell) * (\ell+1) * (j-\ell). -/  
| noncomputable def cc (l j : ℕ) : ℂ :=  
  (-1 : ℂ) ^ (l + 1) * ((j - 1).choose l : ℕ) : ℂ * ((l : ℂ) + 1) * ((j - 1 : ℕ) : ℂ)
```

Commutators and relators.

- cm is the commutator $(a, b) = aba^{-1}b^{-1}$.
- $eqr(a, b) = ab^{-1}$ is the relator that encodes the equation $a = b$.

Lean: cm — library, verified earlier

```

/-- commutator (a,b) = a b a-1 b-1 -/
def cm {c : ℕ → ℕ} (a b : F c) : F c := a * b * a-1 * b-1

```

Lean: eqr — library, verified earlier

```

/-- relator for the equation a = b -/
def eqr {c : ℕ → ℕ} (a b : F c) : F c := a * b-1

```

The elements $\tilde{w}$. w_m and w_w are $\tilde{w}_{-1}(s)$ and $\tilde{w}_{\ell,jk}(s)$, as in (Re:0) and (Im:0).

Lean: w_m — library, verified earlier

```

noncomputable def w_m {c : ℕ → ℕ} (s : ℂ×) : F c :=
  x_m (s : ℂ) * y_m (-(s : ℂ)-1) * x_m (s : ℂ)

```

Lean: w_w — library, verified earlier

```

noncomputable def w_w {c : ℕ → ℕ} (l j k : ℕ) (h : Valid c l j k) (s : ℂ×) : F c :=
  x_x l j k h (s : ℂ) * y_y l j k h (-(s : ℂ)-1) / c_c l j * x_x l j k h (s : ℂ)

```

Relations. The inductive predicate $\text{Rel } c$ has one constructor for each tagged family of relations (H:1a)–(U:4b). Each constructor has the parameters of the source:

- $s, t \in \mathbb{C}^\times$ and $u, v \in \mathbb{C}$;
- membership proofs in E for every index triple that occurs, namely $(j-1, j, k)$ in (U:1a),(U:1b); $(0, j, k)$ in (U:1c),(U:1d); $(1, 2, k)$ and $(0, 2, k)$ in (U:3a),(U:3b); and both (ℓ, j, k) and $(j-1-\ell, j, k)$ in (U:4a),(U:4b), the latter being automatic;
- for (U:2), the side condition $j \neq p \vee k \neq q \vee |\ell - m| > 1$.

A relation $a = b$ is encoded as the relator ab^{-1} , and the commutator relations as the commutators themselves. For example, (Im:5a) appears as $\tilde{w}_{\ell,jk} H_1(s^{j-\ell}) \tilde{w}_{\ell,jk}^{-1} = H_2((s^{\ell+1})^{-1})$, which is the source relation.

Lean: Rel — library, verified earlier

```

/-- The defining relators. -/
inductive Rel (c : ℕ → ℕ) : F c → Prop
| H1a (s t : ℂ×) : Rel c (eqr (h1 s * h1 t) (h1 (s * t)))
| H1b (s t : ℂ×) : Rel c (eqr (h2 s * h2 t) (h2 (s * t)))
| H2 (s t : ℂ×) : Rel c (eqr (h1 s * h2 t) (h2 t * h1 s))
| Re1a (u v : ℂ) : Rel c (eqr (x_m u * x_m v) (x_m (u + v)))
| Re1b (u v : ℂ) : Rel c (eqr (y_m u * y_m v) (y_m (u + v)))
| Re2 (s t : ℂ×) : Rel c (eqr (y_m (-(t : ℂ)) * x_m (s : ℂ) * y_m (t : ℂ))
  (x_m (-(t : ℂ)-1) * y_m (-(t : ℂ) ^ 2) * (s : ℂ) * x_m ((t : ℂ)-1)))
| Re3 (s : ℂ×) : Rel c (eqr (w_m s * w_m 1) (h1 (-s) * h2 (-(s-1))))
| Re4a (u : ℂ) : Rel c (eqr (w_m 1 * x_m u * (w_m 1)-1) (y_m (-u)))
| Re4b (u : ℂ) : Rel c (eqr (w_m 1 * y_m u * (w_m 1)-1) (x_m (-u)))

```

```

| Re5a (s :  $\mathbb{C}^\times$ ) : Rel c (eqr (wm 1 * h1 s * (wm 1)-1) (h2 s))
| Re5b (s :  $\mathbb{C}^\times$ ) : Rel c (eqr (wm 1 * h2 s * (wm 1)-1) (h1 s))
| Re6a (s :  $\mathbb{C}^\times$ ) (u :  $\mathbb{C}$ ) : Rel c (eqr (h1 s * xm u * (h1 s)-1) (xm ((s :  $\mathbb{C}$ ) * u)))
| Re6b (s :  $\mathbb{C}^\times$ ) (u :  $\mathbb{C}$ ) : Rel c (eqr (h2 s * xm u * (h2 s)-1) (xm ((s :  $\mathbb{C}$ )-1 * u)))
| Re6c (s :  $\mathbb{C}^\times$ ) (u :  $\mathbb{C}$ ) : Rel c (eqr (h1 s * ym u * (h1 s)-1) (ym ((s :  $\mathbb{C}$ )-1 * u)))
| Re6d (s :  $\mathbb{C}^\times$ ) (u :  $\mathbb{C}$ ) : Rel c (eqr (h2 s * ym u * (h2 s)-1) (ym ((s :  $\mathbb{C}$ ) * u)))
| Im1a (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u v :  $\mathbb{C}$ ) :
  Rel c (eqr (xx l j k h u * xx l j k h v) (xx l j k h (u + v)))
| Im1b (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u v :  $\mathbb{C}$ ) :
  Rel c (eqr (yy l j k h u * yy l j k h v) (yy l j k h (u + v)))
| Im2 (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (s t :  $\mathbb{C}^\times$ ) :
  Rel c (eqr (yy l j k h (-(t :  $\mathbb{C}$ )) * xx l j k h (s :  $\mathbb{C}$ ) * yy l j k h (t :  $\mathbb{C}$ ))
    (xx l j k h (-(t :  $\mathbb{C}$ )-1 / cc l j) *
      yy l j k h (-(cc l j) * (t :  $\mathbb{C}$ )2 * (s :  $\mathbb{C}$ )) *
      xx l j k h ((t :  $\mathbb{C}$ )-1 / cc l j)))
| Im3 (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (s :  $\mathbb{C}^\times$ ) :
  Rel c (eqr (ww l j k h (s ^ ((l + 1) * (j - 1))) * ww l j k h 1)
    (h1 ((-s) ^ (j - 1)) * h2 ((-s) ^ (l + 1))))
| Im4a (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u :  $\mathbb{C}$ ) :
  Rel c (eqr (ww l j k h 1 * xx l j k h u * (ww l j k h 1)-1) (yy l j k h (-u / cc l j)))
| Im4b (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u :  $\mathbb{C}$ ) :
  Rel c (eqr (ww l j k h 1 * yy l j k h u * (ww l j k h 1)-1) (xx l j k h (-(cc l j) * u)))
| Im5a (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (s :  $\mathbb{C}^\times$ ) :
  Rel c (eqr (ww l j k h 1 * h1 (s ^ (j - 1)) * (ww l j k h 1)-1) (h2 ((s ^ (l + 1))-1)))
| Im5b (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (s :  $\mathbb{C}^\times$ ) :
  Rel c (eqr (ww l j k h 1 * h2 (s ^ (l + 1)) * (ww l j k h 1)-1) (h1 ((s ^ (j - 1))-1)))
| Im6a (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (s :  $\mathbb{C}^\times$ ) (u :  $\mathbb{C}$ ) :
  Rel c (eqr (h1 s * xx l j k h u * (h1 s)-1) (xx l j k h ((s :  $\mathbb{C}$ ) ^ (l + 1) * u)))
| Im6b (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (s :  $\mathbb{C}^\times$ ) (u :  $\mathbb{C}$ ) :
  Rel c (eqr (h2 s * xx l j k h u * (h2 s)-1) (xx l j k h ((s :  $\mathbb{C}$ ) ^ (j - 1) * u)))
| Im6c (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (s :  $\mathbb{C}^\times$ ) (u :  $\mathbb{C}$ ) :
  Rel c (eqr (h1 s * yy l j k h u * (h1 s)-1) (yy l j k h (((s :  $\mathbb{C}$ ) ^ (l + 1))-1 * u)))
| Im6d (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (s :  $\mathbb{C}^\times$ ) (u :  $\mathbb{C}$ ) :
  Rel c (eqr (h2 s * yy l j k h u * (h2 s)-1) (yy l j k h (((s :  $\mathbb{C}$ ) ^ (j - 1))-1 * u)))
| U1a (j k :  $\mathbb{N}$ ) (h : Valid c (j - 1) j k) (u v :  $\mathbb{C}$ ) :
  Rel c (cm (xm u) (xx (j - 1) j k h v))
| U1b (j k :  $\mathbb{N}$ ) (h : Valid c (j - 1) j k) (u v :  $\mathbb{C}$ ) :
  Rel c (cm (ym u) (yy (j - 1) j k h v))
| U1c (j k :  $\mathbb{N}$ ) (h : Valid c 0 j k) (u v :  $\mathbb{C}$ ) :
  Rel c (cm (ym u) (xx 0 j k h v))
| U1d (j k :  $\mathbb{N}$ ) (h : Valid c 0 j k) (u v :  $\mathbb{C}$ ) :
  Rel c (cm (xm u) (yy 0 j k h v))
| U2 (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (m p q :  $\mathbb{N}$ ) (h' : Valid c m p q) (u v :  $\mathbb{C}$ ) :
  (hne : j  $\neq$  p  $\vee$  k  $\neq$  q  $\vee$  1 < Int.natAbs ((l :  $\mathbb{Z}$ ) - (m :  $\mathbb{Z}$ ))) :
  Rel c (cm (xx l j k h u) (yy m p q h' v))
| U3a (k :  $\mathbb{N}$ ) (h : Valid c 1 2 k) (h' : Valid c 0 2 k) (u v :  $\mathbb{C}$ ) :
  Rel c (eqr (cm (xx 1 2 k h u) (yy 0 2 k h' v)) (xm (u * v)))
| U3b (k :  $\mathbb{N}$ ) (h : Valid c 0 2 k) (h' : Valid c 1 2 k) (u v :  $\mathbb{C}$ ) :
  Rel c (eqr (cm (xx 0 2 k h u) (yy 1 2 k h' v)) (ym (u * v)))
| U4a (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (h' : Valid c (j - 1 - 1) j k) (u :  $\mathbb{C}$ ) :
  Rel c (eqr (wm 1 * xx l j k h u * (wm 1)-1)
    (xx (j - 1 - 1) j k h' ((-1 :  $\mathbb{C}$ ) ^ (j - 1 - 1) * u)))
| U4b (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (h' : Valid c (j - 1 - 1) j k) (u :  $\mathbb{C}$ ) :
  Rel c (eqr (wm 1 * yy l j k h u * (wm 1)-1)
    (yy (j - 1 - 1) j k h' ((-1 :  $\mathbb{C}$ ) ^ (j - 1 - 1) * u)))

```

Lean: Rels — library, verified earlier

```
| def Rels (c : ℕ → ℕ) : Set (F c) := {r | Rel c r}
```

The group. Grp c is the presented group $G_c := F(\mathcal{X})/N_{\mathcal{R}}$. For a word $w \in F(\mathcal{X})$ we write $[w] \in G_c$ for its image under the canonical homomorphism $F(\mathcal{X}) \rightarrow G_c$. When c is the coefficient function of J , G_c is $G(\mathbf{m})$.

Lean: Grp — library, verified earlier

```
| /-- The group G(m) = <X | R>. -/
| abbrev Grp (c : ℕ → ℕ) : Type := PresentedGroup (Rels c)
```

Library theorems used. Two library theorems are used below.

- mk_eqr: if $\text{eqr}(a, b) = ab^{-1}$ is a relator, then $[a] = [b]$ in G_c .
- h2_eq_conj: for every $s \in \mathbb{C}^\times$,

$$[H_2(s)] = [\tilde{w}_{-1}] [H_1(s)] [\tilde{w}_{-1}]^{-1}.$$

This follows from (Re:5a) and mk_eqr.

2.2 The subgroups H and U

torusGens c is the set of elements of G_c of the form $[H_1(s)]$ or $[H_2(s)]$ with $s \in \mathbb{C}^\times$.

Lean: torusGens

```
| /-- Generators of the toral subgroup: the images of H_1(s) and H_2(s) in G(m). -/
| def torusGens (c : ℕ → ℕ) : Set (Grp c) :=
| {g | ∃ s : ℂ×, g = PresentedGroup.mk (Rels c) (h1 s) ∨
|   g = PresentedGroup.mk (Rels c) (h2 s)}
```

torusSub c is the subgroup of G_c generated by this set. This is the toral subgroup H .

Lean: torusSub

```
| /-- The toral subgroup H of G(m). -/
| def torusSub (c : ℕ → ℕ) : Subgroup (Grp c) := Subgroup.closure (torusGens c)
```

unipGens c is the set of elements of G_c of one of the forms

$$[X_{-1}(u)], \quad [Y_{-1}(u)], \quad [X_{\ell,jk}(u)], \quad [Y_{\ell,jk}(u)], \quad u \in \mathbb{C}, (\ell, j, k) \in E.$$

Lean: unipGens

```
| /-- The unipotent generators X_{-1}(u), Y_{-1}(u), X_{\ell,jk}(u), Y_{\ell,jk}(u), viewed in G(m). -/
| def unipGens (c : ℕ → ℕ) : Set (Grp c) :=
| {g | (∃ u : ℂ, g = PresentedGroup.mk (Rels c) (xm u)) ∨
|   (∃ u : ℂ, g = PresentedGroup.mk (Rels c) (ym u)) ∨
|   (∃ (l j k : ℕ) (h : Valid c l j k) (u : ℂ),
|     g = PresentedGroup.mk (Rels c) (xx l j k h u)) ∨
|   (∃ (l j k : ℕ) (h : Valid c l j k) (u : ℂ),
|     g = PresentedGroup.mk (Rels c) (yy l j k h u))}
```

unipSub c is the subgroup of G_c generated by this set. This is the unipotent subgroup U .

Lean: unipSub

```

/-- The unipotent subgroup U of G(m). -/
def unipSub (c : ℕ → ℕ) : Subgroup (Grp c) := Subgroup.closure (unipGens c)

```

2.3 The target statement

Lean: target statement gsimple_torus_normalizes_unipotent, fixed before the proof; solution is checked to have exactly this type

```

theorem gsimple_torus_normalizes_unipotent (c : ℕ → ℕ) :
  ∀ h ∈ GM.torusSub c, ∀ u ∈ GM.unipSub c, h * u * h-1 ∈ GM.unipSub c

```

The target statement says the following: for every function $c: \mathbb{N} \rightarrow \mathbb{N}$, every $h \in \text{torusSub } c$ and every $u \in \text{unipSub } c$, we have $huh^{-1} \in \text{unipSub } c$. This is the theorem of the source, with c standing for the coefficient function of J .

The only parameter is c , and the statement carries no hypothesis on it. In particular the hypothesis $c(2) \geq 1$, which the library needs for Theorem 3.3, does not appear. The source statement is therefore the special case where c is the coefficient function of J ; the formal statement holds for every c . The theorem solution below has exactly this statement.

3 Proof

3.1 Overview

The source proof has three steps.

1. By (Re:6a)–(Re:6d) and (Im:6a)–(Im:6d), conjugation by $H_1(s)$ or $H_2(s)$ sends a unipotent generator into U .
2. The source writes $h \in H$ as a product of generators and inverses of generators. It then concludes, by induction on the number of factors, that $hgh^{-1} \in U$ for every generator g of U .
3. The source writes $u \in U$ as a product of generators of U and their inverses. It then uses $huh^{-1} = \prod_i hg_i h^{-1}$ and $hg^{-1}h^{-1} = (hgh^{-1})^{-1}$.

The formal proof proceeds as follows.

- **Step 1.** Lemmas 3.1, 3.2 and 3.5 prove step 1 in a sharper form: the conjugate of a unipotent generator by $H_1(s)$ or $H_2(s)$ is again a unipotent generator.
- **Step 3.** Lemma 3.7 is step 3, proved for an arbitrary group and an arbitrary generating set.
- **Step 2.** Lemma 3.8 is step 2, also proved abstractly.

The order of steps 2 and 3 is exchanged. The formal proof first fixes a single generator t of H and extends “ t conjugates U into U ” from the generators of U to all of U . Only then does it induct over the word for h .

The formal proof deviates from the source on one point. In step 2 the source admits inverses of generators among the factors h_i , but then treats each h_i as some $H_1(s_i)$ or $H_2(s_i)$. This tacitly uses that $H_1(s)^{-1}$ and $H_2(s)^{-1}$ are again of this form. The formal proof establishes this explicitly in two lemmas.

- [Lemma 3.3](#) gives $[H_1(s)]^{-1} = [H_1(s^{-1})]$. It is derived from (H:1a).
- [Lemma 3.4](#) gives the same for H_2 . It is derived from (Re:5a), via the library theorem `h2_eq_conj`, together with [Lemma 3.3](#). Relation (H:1b) would also suffice, but it is not the route taken.

[Lemma 3.6](#) then states that the generating set of H is closed under inverses. This closure is the hypothesis that lets the abstract induction of [Lemma 3.8](#) handle inverse factors.

Lemma 3.1 (`h1_conj_unip`). *Let $c : \mathbb{N} \rightarrow \mathbb{N}$, $s \in \mathbb{C}^\times$, and let $g \in G_c$ belong to `unipGens c`. Then*

$$[H_1(s)] g [H_1(s)]^{-1} \in \text{unipGens } c.$$

Lean: `h1_conj_unip` — proved by AIProver, job 1, session 1

```

theorem h1_conj_unip (c : ℕ → ℕ) (s : ℂ×) (g : GM.Grp c) (hg : g ∈ GM.unipGens c) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 s) * g * (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1
  ∈ GM.unipGens c := by

rcases hg with ⟨⟨u, hg_u⟩ | ⟨u, hg_u⟩ | ⟨1, j, k, h, u, hg_u⟩ | ⟨1, j, k, h, u, hg_u⟩⟩
.
  subst hg_u

  have hrel := GM.Rel.Re6a (c := c) s u
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.h1 s * GM.xm u * (GM.h1 s)-1) =
    PresentedGroup.mk (GM.Rels c) (GM.xm ((s : ℂ) * u)) := by
    simp [GM.eqr] using mk_eqr c (GM.h1 s * GM.xm u * (GM.h1 s)-1) (GM.xm ((s : ℂ) * u)) hrel
  have h_target : PresentedGroup.mk (GM.Rels c) (GM.xm ((s : ℂ) * u)) ∈ GM.unipGens c := by
    refine Or.inl ⟨(s : ℂ) * u, ?_⟩
    rfl
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h1 s) * PresentedGroup.mk (GM.Rels c) (GM.xm u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.xm ((s : ℂ) * u)) := by
    simp [GM.xm, map_mul, map_inv] using h_eq
  rw [h_eq']
  exact h_target
.
  subst hg_u

  have hrel := GM.Rel.Re6c (c := c) s u
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.h1 s * GM.ym u * (GM.h1 s)-1) =
    PresentedGroup.mk (GM.Rels c) (GM.ym (((s : ℂ)-1) * u)) := by
    simp [GM.eqr] using mk_eqr c (GM.h1 s * GM.ym u * (GM.h1 s)-1) (GM.ym (((s : ℂ)-1) * u)) hrel
  have h_target : PresentedGroup.mk (GM.Rels c) (GM.ym (((s : ℂ)-1) * u)) ∈ GM.unipGens c := by
    refine Or.inr (Or.inl ⟨((s : ℂ)-1) * u, ?_⟩)
    rfl
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h1 s) * PresentedGroup.mk (GM.Rels c) (GM.ym u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.ym (((s : ℂ)-1) * u))
  ⇨ := by
    simp [GM.ym, map_mul, map_inv] using h_eq
  rw [h_eq']
  exact h_target
.
  subst hg_u

  have hrel := GM.Rel.Im6a (c := c) 1 j k h s u
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.h1 s * GM.xx 1 j k h u * (GM.h1 s)-1) =

```

```

    PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h ((s : ℂ) ^ (1 + 1) * u)) := by
      simp [GM.eqr] using mk_eqr c (GM.h1 s * GM.xx 1 j k h u * (GM.h1 s)-1) (GM.xx 1 j k h ((s : ℂ) ^ (1 +
↪ 1) * u)) hrel
    have h_target : PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h ((s : ℂ) ^ (1 + 1) * u)) ∈ GM.unipGens c
↪ := by
      refine Or.inr (Or.inr (Or.inl ⟨1, j, k, h, ((s : ℂ) ^ (1 + 1) * u), ?_⟩))
      rfl
    have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h1 s) * PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h u) *
      (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h ((s : ℂ) ^
↪ (1 + 1) * u)) := by
      simp [GM.xx, map_mul, map_inv] using h_eq
      rw [h_eq']
      exact h_target
  .
  subst hg_u

  have hrel := GM.Rel.Im6c (c := c) 1 j k h s u
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.h1 s * GM.yy 1 j k h u * (GM.h1 s)-1) =
    PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (1 + 1))-1 * u)) := by
      simp [GM.eqr] using mk_eqr c (GM.h1 s * GM.yy 1 j k h u * (GM.h1 s)-1) (GM.yy 1 j k h (((s : ℂ) ^ (1
↪ + 1))-1 * u)) hrel
  have h_target : PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (1 + 1))-1 * u)) ∈ GM.unipGens
↪ c := by
    refine Or.inr (Or.inr (Or.inr ⟨1, j, k, h, (((s : ℂ) ^ (1 + 1))-1 * u), ?_⟩))
    rfl
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h1 s) * PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ)
↪ ^ (1 + 1))-1 * u)) := by
    simp [GM.yy, map_mul, map_inv] using h_eq
    rw [h_eq']
    exact h_target

```

Proof. There are four cases, according to the form of g . In each case the relevant relator lies in $\mathcal{R}$, so mk_eqr gives an equality in G_c . Its left-hand side equals $[H_1(s)] g [H_1(s)]^{-1}$, because $w \mapsto [w]$ is a homomorphism.

- If $g = [X_{-1}(u)]$, then (Re:6a) gives $[H_1(s)X_{-1}(u)H_1(s)^{-1}] = [X_{-1}(su)]$.
- If $g = [Y_{-1}(u)]$, then (Re:6c) gives $[Y_{-1}(s^{-1}u)]$.
- If $g = [X_{\ell,jk}(u)]$ with $(\ell, j, k) \in E$, then (Im:6a) gives $[X_{\ell,jk}(s^{\ell+1}u)]$.
- If $g = [Y_{\ell,jk}(u)]$ with $(\ell, j, k) \in E$, then (Im:6c) gives $[Y_{\ell,jk}((s^{\ell+1})^{-1}u)]$.

In each case the right-hand side is the image of a unipotent generator, with the same index $(\ell, j, k) \in E$ in the last two cases. It therefore lies in $\text{unipGens } c$. $\square$

Lemma 3.2 (h2_conj_unip). *Let $c: \mathbb{N} \rightarrow \mathbb{N}$, $s \in \mathbb{C}^\times$, and let $g \in G_c$ belong to $\text{unipGens } c$. Then*

$$[H_2(s)] g [H_2(s)]^{-1} \in \text{unipGens } c.$$

Lean: h2_conj_unip — proved by AIProver, job 1, session 0

```

theorem h2_conj_unip (c : ℕ → ℕ) (s : ℂ×) (g : GM.Grp c) (hg : g ∈ GM.unipGens c) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s) * g * (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1
    ∈ GM.unipGens c := by
  rcases hg with (⟨u, hg'⟩ | ⟨u, hg'⟩ | ⟨1, j, k, h, u, hg'⟩ | ⟨1, j, k, h, u, hg'⟩)
  .
  subst hg'
  have h_rel := GM.Rel.Re6b (c := c) s u
  have h_eq := mk_eqr c (GM.h2 s * GM.xm u * (GM.h2 s)-1) (GM.xm ((s : ℂ)-1 * u)) h_rel
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h2 s) * PresentedGroup.mk (GM.Rels c) (GM.xm u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.xm ((s : ℂ)-1 * u))
  ↪ := by
    simpa [PresentedGroup.mk, map_mul, map_inv] using h_eq
    rw [h_eq']
    exact Set.mem_setOf.mpr (Or.inl ((s : ℂ)-1 * u, rfl))
  .
  subst hg'
  have h_rel := GM.Rel.Re6d (c := c) s u
  have h_eq := mk_eqr c (GM.h2 s * GM.ym u * (GM.h2 s)-1) (GM.ym ((s : ℂ) * u)) h_rel
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h2 s) * PresentedGroup.mk (GM.Rels c) (GM.ym u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.ym ((s : ℂ) * u)) := by
    simpa [PresentedGroup.mk, map_mul, map_inv] using h_eq
    rw [h_eq']
    exact Set.mem_setOf.mpr (Or.inr (Or.inl ((s : ℂ) * u, rfl)))
  .
  subst hg'
  have h_rel := GM.Rel.Im6b (c := c) 1 j k h s u
  have h_eq := mk_eqr c (GM.h2 s * GM.xx 1 j k h u * (GM.h2 s)-1) (GM.xx 1 j k h (((s : ℂ) ^ (j - 1)) *
  ↪ u)) h_rel
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h2 s) * PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (((s : ℂ)
  ↪ ^ (j - 1)) * u)) := by
    simpa [PresentedGroup.mk, map_mul, map_inv] using h_eq
    rw [h_eq']
    exact Set.mem_setOf.mpr (Or.inr (Or.inr (Or.inl ⟨1, j, k, h, ((s : ℂ) ^ (j - 1)) * u, rfl⟩)))
  .
  subst hg'
  have h_rel := GM.Rel.Im6d (c := c) 1 j k h s u
  have h_eq := mk_eqr c (GM.h2 s * GM.yy 1 j k h u * (GM.h2 s)-1) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1
  ↪ * u)) h_rel
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h2 s) * PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ)
  ↪ ^ (j - 1))-1 * u)) := by
    simpa [PresentedGroup.mk, map_mul, map_inv] using h_eq
    rw [h_eq']
    exact Set.mem_setOf.mpr (Or.inr (Or.inr (Or.inr ⟨1, j, k, h, (((s : ℂ) ^ (j - 1))-1 * u, rfl⟩)))

```

Proof. The argument is the same as for [Lemma 3.1](#), using (Re:6b), (Re:6d), (Im:6b), (Im:6d) with mk_eqr. The conjugate equals, respectively,

$$\left[X_{-1}(s^{-1}u) \right], \quad \left[Y_{-1}(su) \right], \quad \left[X_{\ell,jk}(s^{j-\ell}u) \right], \quad \left[Y_{\ell,jk}((s^{j-\ell})^{-1}u) \right].$$

Each of these is the image of a unipotent generator. □

Lemma 3.3 (h1_inv_mk). *For every $c: \mathbb{N} \rightarrow \mathbb{N}$ and every $s \in \mathbb{C}^\times$, we have $[H_1(s^{-1})] = [H_1(s)]^{-1}$ in G_c .*

Lean: h1_inv_mk — proved by AIProver, job 1, session 0

```

theorem h1_inv_mk (c : ℕ → ℕ) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 s-1) = (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 := by
  let φ := PresentedGroup.mk (GM.Rels c)
  have hφ_mul : ∀ x y : GM.F c, φ (x * y) = φ x * φ y := by
    intro x y; exact MonoidHom.map_mul φ x y
  have hφ_inv : ∀ x : GM.F c, φ (x-1) = (φ x)-1 := by
    intro x; exact MonoidHom.map_inv φ x
  have h1_one : φ (GM.h1 1) = 1 := by
    have hRel : GM.Rel c (GM.eqr (GM.h1 1 * GM.h1 1) (GM.h1 (1 * 1))) :=
      GM.Rel.H1a (c := c) 1 1
    have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)
    have hcalc : φ (GM.eqr (GM.h1 1 * GM.h1 1) (GM.h1 (1 * 1))) =
      φ (GM.h1 1 * GM.h1 1) * (φ (GM.h1 (1 * 1)))-1 := by
      simp [GM.eqr, hφ_mul, hφ_inv]
    rw [hcalc] at h_one
    rw [mul_one] at h_one
    rw [hφ_mul] at h_one
  calc
    φ (GM.h1 1) = (φ (GM.h1 1) * φ (GM.h1 1)) * (φ (GM.h1 1))-1 := by group
    _ = 1 := h_one
  have hRel : GM.Rel c (GM.eqr (GM.h1 s * GM.h1 s-1) (GM.h1 (s * s-1))) :=
    GM.Rel.H1a (c := c) s s-1
  have hRel' : GM.Rel c (GM.eqr (GM.h1 s * GM.h1 s-1) (GM.h1 1)) := by
    simpa using hRel
  have h_eq := mk_eqr c (GM.h1 s * GM.h1 s-1) (GM.h1 1) hRel'
  calc
    φ (GM.h1 s-1) = 1 * φ (GM.h1 s-1) := by simp
    _ = ((φ (GM.h1 s))-1 * φ (GM.h1 s)) * φ (GM.h1 s-1) := by simp
    _ = (φ (GM.h1 s))-1 * (φ (GM.h1 s) * φ (GM.h1 s-1)) := by group
    _ = (φ (GM.h1 s))-1 * φ (GM.h1 s * GM.h1 s-1) := by rw [hφ_mul]
    _ = (φ (GM.h1 s))-1 * φ (GM.h1 1) := by rw [h_eq]
    _ = (φ (GM.h1 s))-1 * 1 := by rw [h1_one]
    _ = (φ (GM.h1 s))-1 := by simp

```

Proof. Step 1: $[H_1(1)] = 1$. Relation (H:1a) with $s = t = 1$ gives $[H_1(1)] [H_1(1)] = [H_1(1)]$. Cancelling one factor yields $[H_1(1)] = 1$.

Step 2. Relation (H:1a) with $t = s^{-1}$, together with $ss^{-1} = 1$ in $\mathbb{C}^\times$ and mk_eqr , gives $[H_1(s)] [H_1(s^{-1})] = [H_1(1)] = 1$. Multiplying on the left by $[H_1(s)]^{-1}$ gives the claim. $\square$

Lemma 3.4 (h2_inv_mk). *For every $c: \mathbb{N} \rightarrow \mathbb{N}$ and every $s \in \mathbb{C}^\times$, we have $[H_2(s^{-1})] = [H_2(s)]^{-1}$ in G_c .*

Lean: h2_inv_mk — proved by AIProver, job 1, session 0

```

theorem h2_inv_mk (c : ℕ → ℕ) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s-1) = (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 := by
  rw [h2_eq_conj c s-1, h2_eq_conj c s]
  simp
  rw [h1_inv_mk c s]
  group

```

Proof. Write $w = [\tilde{w}_{-1}]$. By the library theorem `h2_eq_conj`, which follows from (Re:5a), applied to s^{-1} and to s , and by [Lemma 3.3](#),

$$\left[H_2(s^{-1}) \right] = w \left[H_1(s^{-1}) \right] w^{-1} = w \left[H_1(s) \right]^{-1} w^{-1} = (w \left[H_1(s) \right] w^{-1})^{-1} = \left[H_2(s) \right]^{-1}. \quad \square$$

Lemma 3.5 (`torusGens_conj`). *Let $c: \mathbb{N} \rightarrow \mathbb{N}$. For every $t \in \text{torusGens } c$ and every $g \in \text{unipGens } c$, we have $tgt^{-1} \in \text{unipGens } c$.*

Lean: `torusGens_conj` — proved by AIProver, job 1, session 1

```

theorem torusGens_conj (c : ℕ → ℕ) :
  ∀ t ∈ GM.torusGens c, ∀ g ∈ GM.unipGens c, t * g * t⁻¹ ∈ GM.unipGens c := by
  intro t ht g hg
  rcases ht with ⟨s, hs⟩
  rcases hs with ⟨hs | hs⟩
  ·
    rw [hs]
    exact h1_conj_unip c s g hg
  ·
    rw [hs]
    exact h2_conj_unip c s g hg

```

Proof. By definition $t = [H_1(s)]$ or $t = [H_2(s)]$ for some $s \in \mathbb{C}^\times$. The claim is then [Lemma 3.1](#) or [Lemma 3.2](#) respectively. $\square$

Lemma 3.6 (`torusGens_inv`). *Let $c: \mathbb{N} \rightarrow \mathbb{N}$. If $t \in \text{torusGens } c$, then $t^{-1} \in \text{torusGens } c$.*

Lean: `torusGens_inv` — proved by AIProver, job 1, session 0

```

theorem torusGens_inv (c : ℕ → ℕ) :
  ∀ t ∈ GM.torusGens c, t⁻¹ ∈ GM.torusGens c := by
  intro t ht
  rcases ht with ⟨s, hs | hs⟩
  · rw [hs]
    refine ⟨s⁻¹, Or.inl ?_⟩
    rw [h1_inv_mk c s]
  · rw [hs]
    refine ⟨s⁻¹, Or.inr ?_⟩
    rw [h2_inv_mk c s]

```

Proof. If $t = [H_1(s)]$, then $t^{-1} = [H_1(s^{-1})]$ by [Lemma 3.3](#). If $t = [H_2(s)]$, then $t^{-1} = [H_2(s^{-1})]$ by [Lemma 3.4](#). In both cases $s^{-1} \in \mathbb{C}^\times$, so $t^{-1} \in \text{torusGens } c$. $\square$

Lemma 3.7 (`conj_closure_of_gens`). *Let G be a group, $T \subseteq G$ a subset and $x \in G$. Suppose $xgx^{-1} \in \langle T \rangle$ for every $g \in T$. Then $xux^{-1} \in \langle T \rangle$ for every $u \in \langle T \rangle$.*

Lean: conj_closure_of_gens — proved by AIProver, job 1, session 1

```

theorem conj_closure_of_gens {G : Type*} [Group G] (T : Set G) (x : G)
  (hx : ∀ g ∈ T, x * g * x-1 ∈ Subgroup.closure T) :
  ∀ u ∈ Subgroup.closure T, x * u * x-1 ∈ Subgroup.closure T := by
  intro u hu
  induction hu using Subgroup.closure_induction with
  | mem g hg =>
    exact hx g hg
  | one =>
    simp
  | mul a b ha hb iha ihb =>
    have h : x * (a * b) * x-1 = (x * a * x-1) * (x * b * x-1) := by group
    rw [h]
    exact Subgroup.mul_mem _ iha ihb
  | inv a ha iha =>
    have hxa : x * a-1 * x-1 = (x * a * x-1)-1 := by group
    rw [hxa]
    exact Subgroup.inv_mem _ iha

```

Proof. We use induction over the generation of $\langle T \rangle$. Every element of $\langle T \rangle$ is obtained from elements of T and from 1 by products and inverses, so it suffices to check that the property “ $xux^{-1} \in \langle T \rangle$ ” is preserved by these operations.

- For $u \in T$ it holds by hypothesis.
- For $u = 1$ we have $x1x^{-1} = 1 \in \langle T \rangle$.
- If it holds for a and b , then $x(ab)x^{-1} = (xax^{-1})(xbx^{-1}) \in \langle T \rangle$.
- If it holds for a , then $xa^{-1}x^{-1} = (xax^{-1})^{-1} \in \langle T \rangle$. □

Lemma 3.8 (closure_normalizes). *Let G be a group and $S, T \subseteq G$. Assume:*

1. $t^{-1} \in S$ for every $t \in S$;
2. $tut^{-1} \in \langle T \rangle$ for every $t \in S$ and every $u \in \langle T \rangle$.

Then $huh^{-1} \in \langle T \rangle$ for every $h \in \langle S \rangle$ and every $u \in \langle T \rangle$.

Lean: closure_normalizes — proved by AIProver, job 2, session 4

```

theorem closure_normalizes {G : Type*} [Group G] (S T : Set G)
  (hinv : ∀ t ∈ S, t-1 ∈ S)
  (hconj : ∀ t ∈ S, ∀ u ∈ Subgroup.closure T, t * u * t-1 ∈ Subgroup.closure T) :
  ∀ h ∈ Subgroup.closure S, ∀ u ∈ Subgroup.closure T, h * u * h-1 ∈ Subgroup.closure T := by
  intro h hh
  induction hh using Subgroup.closure_induction_left with
  | one =>
    intro u hu
    simp using hu
  | mul_left x hx y hy ih =>
    intro u hu
    have hcalc : (x * y) * u * (x * y)-1 = x * (y * u * y-1) * x-1 := by
      simp [mul_assoc]
    rw [hcalc]

```

```

    exact hconj x hx (y * u * y-1) (ih u hu)
| inv_mul_cancel x hx y hy ih =>
  intro u hu
  have hcalc : (x-1 * y) * u * (x-1 * y)-1 = x-1 * (y * u * y-1) * x := by
    simp [mul_assoc]
  rw [hcalc]
  have hcalc2 : x-1 * (y * u * y-1) * x = x-1 * (y * u * y-1) * (x-1)-1 := by
    simp
  rw [hcalc2]
  have hx_inv : x-1 ∈ S := hinv x hx
  exact hconj x-1 hx_inv (y * u * y-1) (ih u hu)

```

Proof. Let $P(h)$ be the statement “ $huh^{-1} \in \langle T \rangle$ for all $u \in \langle T \rangle$ ”. Every element of $\langle S \rangle$ is obtained from 1 by successive left multiplications by elements $x \in S$ or by inverses x^{-1} of elements $x \in S$. It therefore suffices to prove three things.

- $P(1)$ holds, since $1u1^{-1} = u$.
- If $x \in S$, $y \in \langle S \rangle$ and $P(y)$, then $P(xy)$. Indeed

$$(xy)u(xy)^{-1} = x(yuy^{-1})x^{-1}.$$

Here $yuy^{-1} \in \langle T \rangle$ by $P(y)$, so hypothesis (2) applied to x gives the claim.

- If $x \in S$, $y \in \langle S \rangle$ and $P(y)$, then $P(x^{-1}y)$. Indeed

$$(x^{-1}y)u(x^{-1}y)^{-1} = x^{-1}(yuy^{-1})(x^{-1})^{-1}.$$

By hypothesis (1), $x^{-1} \in S$, so hypothesis (2) applied to x^{-1} and to $yuy^{-1} \in \langle T \rangle$ gives the claim. $\square$

3.2 Proof of the theorem

Lean: solution — main proof from the captain's sketch

```

theorem solution (c : ℕ → ℕ) :
  ∀ h ∈ GM.torusSub c, ∀ u ∈ GM.unipSub c, h * u * h-1 ∈ GM.unipSub c := by
  intro h hh u hu
  exact closure_normalizes (GM.torusGens c) (GM.unipGens c) (torusGens_inv c)
    (fun t ht => conj_closure_of_gens (GM.unipGens c) t
      (fun g hg => Subgroup.subset_closure (torusGens_conj c t ht g hg))) h hh u hu

```

Proof. Fix $c : \mathbb{N} \rightarrow \mathbb{N}$. Apply [Lemma 3.8](#) in the group G_c with

$$S = \text{torusGens } c, \quad T = \text{unipGens } c.$$

Then $\langle S \rangle = \text{torusSub } c$ and $\langle T \rangle = \text{unipSub } c$. The two hypotheses are checked as follows.

1. The closure of S under inverses is [Lemma 3.6](#).
2. Let $t \in S$. For every $g \in T$, [Lemma 3.5](#) gives $tgt^{-1} \in T \subseteq \langle T \rangle$. So [Lemma 3.7](#), applied with $x = t$, gives $tut^{-1} \in \langle T \rangle$ for every $u \in \langle T \rangle$.

[Lemma 3.8](#) then yields $huh^{-1} \in \text{unipSub } c$ for all $h \in \text{torusSub } c$ and all $u \in \text{unipSub } c$. This is the target statement. $\square$

4 Independent review

A referee model (claude-opus-5-5), which took no part in the formalization or the proof, compared the verified Lean file with the source. Verdict: **FAITHFUL**.

Definitions. I checked each object in the GM namespace against the source.

- `Valid` encodes the index set E : it requires $\ell < j$ and $1 \leq k \leq c(j)$. Since $\ell \geq 0$, this forces $j \geq 1$. The function $c : \mathbb{N} \rightarrow \mathbb{N}$ is a parameter, as the formalization note allows.
- `Gen` has one constructor for each family of symbols in $\mathcal{X}$. These are $H_1(s)$ and $H_2(s)$ for $s \in \mathbb{C}^\times$; $X_{-1}(u)$ and $Y_{-1}(u)$ for $u \in \mathbb{C}$; and $X_{\ell,jk}(u)$ and $Y_{\ell,jk}(u)$, which carry a proof of `Valid`. That proof is a `Prop`, so it adds no spurious distinct generators.
- `cc` is $(-1)^{\ell+1} \binom{j-1}{\ell} (\ell+1)(j-\ell)$. The truncated subtraction $j-\ell$ is harmless because $\ell < j$.
- `wm` matches (Re:0) and `ww` matches (Im:0).
- `cm` is $aba^{-1}b^{-1}$ and `eqr` is ab^{-1} , as in the source.
- `Rel` has one constructor for every tagged relation, from (H:1a) through (U:4b). I compared each one with the source, including:
 - the scalars in (Re:2), (Im:2), (Re:3) and (Im:3);
 - the inverted exponents in (Im:5a), (Im:5b), (Im:6c) and (Im:6d);
 - the index $j-1$ in (U:1a) and (U:1b);
 - the condition $|\ell - m| > 1$ in (U:2), written with `Int.natAbs`;
 - the sign $(-1)^{j-\ell-1}$ in (U:4a) and (U:4b).

All agree. In (U:4a) and (U:4b) the extra hypothesis that $(j-1-\ell, j, k)$ is valid follows automatically from validity of (ℓ, j, k) , so it does not weaken the relations.

- `Grp` is the presented group $F(\mathcal{X})/N_{\mathcal{R}}$.
- `torusSub` is the subgroup generated by the images of all $H_1(s)$ and $H_2(s)$; this is H .
- `unipSub` is the subgroup generated by the images of all four families of unipotent generators; this is U .

I found no discrepancies.

Statement. The target says: for every $c : \mathbb{N} \rightarrow \mathbb{N}$, every $h \in \text{torusSub } c$ and every $u \in \text{unipSub } c$, the element huh^{-1} lies in `unipSub` c . This is exactly the source conclusion.

No hypothesis on c is needed. To recover the source statement, instantiate $c(j)$ as the coefficient of q^j in J for $j \geq 1$. These coefficients are nonnegative integers, and $c(0) = 0$ is consistent with the source. Only $j \geq 1$ is ever used.

The result is not vacuous. The relations are those of the source, so the group is not degenerate by construction, and H and U are the genuine subgroups described there.

Proof. The lemmas from `mk_rel_one` through `gsimple_g_m_perfect` are carried over from the Theorem 3.3 development. `solution` does not use them.

The relevant part of the proof runs as follows.

- `h1_conj_unip` and `h2_conj_unip` show that conjugating a unipotent generator by $H_1(s)$ or $H_2(s)$ gives another unipotent generator. They use (Re:6a) to (Re:6d) and (Im:6a) to (Im:6d). This is the first paragraph of the source proof.
- `torusGens_inv` shows that the generating set of H is closed under inverses. It relies on two auxiliary lemmas:
 - `h1_inv_mk` proves $H_1(s^{-1}) = H_1(s)^{-1}$ from (H:1a);
 - `h2_inv_mk` proves $H_2(s^{-1}) = H_2(s)^{-1}$ by writing H_2 as a conjugate of H_1 via (Re:5a).

The source silently allows "generators or their inverses". The formal proof justifies inverses explicitly, using (Re:5a) rather than the analogous (H:1b) for H_2 . This is a legitimate deviation.

- `conj_closure_of_gens` carries out the source's last paragraph. By closure induction, conjugation by a fixed x that maps generators into U maps all of U into U ; inverses of generators are handled as in the source.
- `closure_normalizes` carries out the source's induction over a word $h = h_1 \cdots h_m$, using `Subgroup.closure_induction_left`.

All of these lemmas state exactly the step they represent.

Concerns.

- The coefficients $c(j)$ of J are a free parameter. The theorem is proved for every c , so it covers the actual coefficients.
- The file contains a large block of unused lemmas from the Theorem 3.3 development. They are harmless to this result.

A Complete Lean file

The verified file, as checked by Lean.

```
import Mathlib
set_option autoImplicit false

set_option autoImplicit false

-- From run srv_20261004_123235_GSimple_Theorem_3_3 (GSimple_Theorem_3_3), proved.

namespace GM

/-- Index set E: ( $\ell, j, k$ ) with  $0 \leq \ell < j$  and  $1 \leq k \leq c(j)$ . -/
def Valid (c : ℕ → ℕ) (l j k : ℕ) : Prop := l < j ∧ 1 ≤ k ∧ k ≤ c j

/-- The generating symbols. -/
inductive Gen (c : ℕ → ℕ) : Type
| H1 : ℂ× → Gen c
```

```

| H2 :  $\mathbb{C}^\times \rightarrow \text{Gen } c$ 
| Xm :  $\mathbb{C} \rightarrow \text{Gen } c$ 
| Ym :  $\mathbb{C} \rightarrow \text{Gen } c$ 
| X (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u :  $\mathbb{C}$ ) : Gen c
| Y (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u :  $\mathbb{C}$ ) : Gen c

abbrev F (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) := FreeGroup (Gen c)

def h1 {c :  $\mathbb{N} \rightarrow \mathbb{N}$ } (s :  $\mathbb{C}^\times$ ) : F c := FreeGroup.of (Gen.H1 s)
def h2 {c :  $\mathbb{N} \rightarrow \mathbb{N}$ } (s :  $\mathbb{C}^\times$ ) : F c := FreeGroup.of (Gen.H2 s)
def xm {c :  $\mathbb{N} \rightarrow \mathbb{N}$ } (u :  $\mathbb{C}$ ) : F c := FreeGroup.of (Gen.Xm u)
def ym {c :  $\mathbb{N} \rightarrow \mathbb{N}$ } (u :  $\mathbb{C}$ ) : F c := FreeGroup.of (Gen.Ym u)
def xx {c :  $\mathbb{N} \rightarrow \mathbb{N}$ } (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u :  $\mathbb{C}$ ) : F c :=
  FreeGroup.of (Gen.X l j k h u)
def yy {c :  $\mathbb{N} \rightarrow \mathbb{N}$ } (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u :  $\mathbb{C}$ ) : F c :=
  FreeGroup.of (Gen.Y l j k h u)

/--  $c_{\ell j} = (-1)^{\ell+1} * \text{binom}(j-1, \ell) * (\ell+1) * (j-\ell)$ . -/
noncomputable def cc (l j :  $\mathbb{N}$ ) :  $\mathbb{C}$  :=
  (-1 :  $\mathbb{C}$ ) ^ (l + 1) * ((j - 1).choose l :  $\mathbb{N}$ ) :  $\mathbb{C}$  * ((1 :  $\mathbb{C}$ ) + 1) * ((j - 1 :  $\mathbb{N}$ ) :  $\mathbb{C}$ )

/-- commutator (a,b) = a b a-1 b-1 -/
def cm {c :  $\mathbb{N} \rightarrow \mathbb{N}$ } (a b : F c) : F c := a * b * a-1 * b-1

/-- relator for the equation a = b -/
def eqr {c :  $\mathbb{N} \rightarrow \mathbb{N}$ } (a b : F c) : F c := a * b-1

noncomputable def wm {c :  $\mathbb{N} \rightarrow \mathbb{N}$ } (s :  $\mathbb{C}^\times$ ) : F c :=
  xm (s :  $\mathbb{C}$ ) * ym (-(s :  $\mathbb{C}$ )-1) * xm (s :  $\mathbb{C}$ )

noncomputable def ww {c :  $\mathbb{N} \rightarrow \mathbb{N}$ } (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (s :  $\mathbb{C}^\times$ ) : F c :=
  xx l j k h (s :  $\mathbb{C}$ ) * yy l j k h (-(s :  $\mathbb{C}$ )-1) / cc l j * xx l j k h (s :  $\mathbb{C}$ )

/-- The defining relators. -/
inductive Rel (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) : F c → Prop
| H1a (s t :  $\mathbb{C}^\times$ ) : Rel c (eqr (h1 s * h1 t) (h1 (s * t)))
| H1b (s t :  $\mathbb{C}^\times$ ) : Rel c (eqr (h2 s * h2 t) (h2 (s * t)))
| H2 (s t :  $\mathbb{C}^\times$ ) : Rel c (eqr (h1 s * h2 t) (h2 t * h1 s))
| Re1a (u v :  $\mathbb{C}$ ) : Rel c (eqr (xm u * xm v) (xm (u + v)))
| Re1b (u v :  $\mathbb{C}$ ) : Rel c (eqr (ym u * ym v) (ym (u + v)))
| Re2 (s t :  $\mathbb{C}^\times$ ) : Rel c (eqr (ym (-(t :  $\mathbb{C}$ )) * xm (s :  $\mathbb{C}$ ) * ym (t :  $\mathbb{C}$ ))
  (xm (-(t :  $\mathbb{C}$ )-1) * ym (-(t :  $\mathbb{C}$ ) ^ 2 * (s :  $\mathbb{C}$ )) * xm ((t :  $\mathbb{C}$ )-1)))
| Re3 (s :  $\mathbb{C}^\times$ ) : Rel c (eqr (wm s * wm 1) (h1 (-s) * h2 (-(s-1))))
| Re4a (u :  $\mathbb{C}$ ) : Rel c (eqr (wm 1 * xm u * (wm 1)-1) (ym (-u)))
| Re4b (u :  $\mathbb{C}$ ) : Rel c (eqr (wm 1 * ym u * (wm 1)-1) (xm (-u)))
| Re5a (s :  $\mathbb{C}^\times$ ) : Rel c (eqr (wm 1 * h1 s * (wm 1)-1) (h2 s))
| Re5b (s :  $\mathbb{C}^\times$ ) : Rel c (eqr (wm 1 * h2 s * (wm 1)-1) (h1 s))
| Re6a (s :  $\mathbb{C}^\times$ ) (u :  $\mathbb{C}$ ) : Rel c (eqr (h1 s * xm u * (h1 s)-1) (xm ((s :  $\mathbb{C}$ ) * u)))
| Re6b (s :  $\mathbb{C}^\times$ ) (u :  $\mathbb{C}$ ) : Rel c (eqr (h2 s * xm u * (h2 s)-1) (xm ((s :  $\mathbb{C}$ )-1 * u)))
| Re6c (s :  $\mathbb{C}^\times$ ) (u :  $\mathbb{C}$ ) : Rel c (eqr (h1 s * ym u * (h1 s)-1) (ym ((s :  $\mathbb{C}$ )-1 * u)))
| Re6d (s :  $\mathbb{C}^\times$ ) (u :  $\mathbb{C}$ ) : Rel c (eqr (h2 s * ym u * (h2 s)-1) (ym ((s :  $\mathbb{C}$ ) * u)))
| Im1a (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u v :  $\mathbb{C}$ ) :
  Rel c (eqr (xx l j k h u * xx l j k h v) (xx l j k h (u + v)))
| Im1b (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (u v :  $\mathbb{C}$ ) :
  Rel c (eqr (yy l j k h u * yy l j k h v) (yy l j k h (u + v)))
| Im2 (l j k :  $\mathbb{N}$ ) (h : Valid c l j k) (s t :  $\mathbb{C}^\times$ ) :
  Rel c (eqr (yy l j k h (-(t :  $\mathbb{C}$ )) * xx l j k h (s :  $\mathbb{C}$ ) * yy l j k h (t :  $\mathbb{C}$ ))
  (xx l j k h (-(t :  $\mathbb{C}$ )-1) / cc l j *
  yy l j k h (-(cc l j) * (t :  $\mathbb{C}$ ) ^ 2 * (s :  $\mathbb{C}$ )) *

```

```

      xx 1 j k h ((t : ℂ)-1 / cc 1 j)))
| Im3 (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) :
  Rel c (eqr (ww 1 j k h (s ^ ((1 + 1) * (j - 1))) * ww 1 j k h 1)
    (h1 ((-s) ^ (j - 1)) * h2 ((-s) ^ (1 + 1))))))
| Im4a (1 j k : ℕ) (h : Valid c 1 j k) (u : ℂ) :
  Rel c (eqr (ww 1 j k h 1 * xx 1 j k h u * (ww 1 j k h 1)-1) (yy 1 j k h (-u / cc 1 j)))
| Im4b (1 j k : ℕ) (h : Valid c 1 j k) (u : ℂ) :
  Rel c (eqr (ww 1 j k h 1 * yy 1 j k h u * (ww 1 j k h 1)-1) (xx 1 j k h (-cc 1 j * u)))
| Im5a (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) :
  Rel c (eqr (ww 1 j k h 1 * h1 (s ^ (j - 1)) * (ww 1 j k h 1)-1) (h2 ((s ^ (1 + 1))-1)))
| Im5b (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) :
  Rel c (eqr (ww 1 j k h 1 * h2 (s ^ (1 + 1)) * (ww 1 j k h 1)-1) (h1 ((s ^ (j - 1))-1)))
| Im6a (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h1 s * xx 1 j k h u * (h1 s)-1) (xx 1 j k h ((s : ℂ) ^ (1 + 1) * u)))
| Im6b (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h2 s * xx 1 j k h u * (h2 s)-1) (xx 1 j k h ((s : ℂ) ^ (j - 1) * u)))
| Im6c (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h1 s * yy 1 j k h u * (h1 s)-1) (yy 1 j k h (((s : ℂ) ^ (1 + 1))-1 * u)))
| Im6d (1 j k : ℕ) (h : Valid c 1 j k) (s : ℂ×) (u : ℂ) :
  Rel c (eqr (h2 s * yy 1 j k h u * (h2 s)-1) (yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * u)))
| U1a (j k : ℕ) (h : Valid c (j - 1) j k) (u v : ℂ) :
  Rel c (cm (xm u) (xx (j - 1) j k h v))
| U1b (j k : ℕ) (h : Valid c (j - 1) j k) (u v : ℂ) :
  Rel c (cm (ym u) (yy (j - 1) j k h v))
| U1c (j k : ℕ) (h : Valid c 0 j k) (u v : ℂ) :
  Rel c (cm (ym u) (xx 0 j k h v))
| U1d (j k : ℕ) (h : Valid c 0 j k) (u v : ℂ) :
  Rel c (cm (xm u) (yy 0 j k h v))
| U2 (1 j k : ℕ) (h : Valid c 1 j k) (m p q : ℕ) (h' : Valid c m p q) (u v : ℂ) :
  (hne : j ≠ p ∨ k ≠ q ∨ 1 < Int.natAbs ((1 : ℤ) - (m : ℤ))) :
  Rel c (cm (xx 1 j k h u) (yy m p q h' v))
| U3a (k : ℕ) (h : Valid c 1 2 k) (h' : Valid c 0 2 k) (u v : ℂ) :
  Rel c (eqr (cm (xx 1 2 k h u) (yy 0 2 k h' v)) (xm (u * v)))
| U3b (k : ℕ) (h : Valid c 0 2 k) (h' : Valid c 1 2 k) (u v : ℂ) :
  Rel c (eqr (cm (xx 0 2 k h u) (yy 1 2 k h' v)) (ym (u * v)))
| U4a (1 j k : ℕ) (h : Valid c 1 j k) (h' : Valid c (j - 1 - 1) j k) (u : ℂ) :
  Rel c (eqr (wm 1 * xx 1 j k h u * (wm 1)-1)
    (xx (j - 1 - 1) j k h' ((-1 : ℂ) ^ (j - 1 - 1) * u)))
| U4b (1 j k : ℕ) (h : Valid c 1 j k) (h' : Valid c (j - 1 - 1) j k) (u : ℂ) :
  Rel c (eqr (wm 1 * yy 1 j k h u * (wm 1)-1)
    (yy (j - 1 - 1) j k h' ((-1 : ℂ) ^ (j - 1 - 1) * u)))

def Rels (c : ℕ → ℕ) : Set (F c) := {r | Rel c r}

/-- The group G(m) = ⟨X | R⟩. -/
abbrev Grp (c : ℕ → ℕ) : Type := PresentedGroup (Rels c)

end GM

theorem mk_rel_one (c : ℕ → ℕ) (r : GM.F c) (h : GM.Rel c r) :
  PresentedGroup.mk (GM.Rels c) r = 1 := by
  apply PresentedGroup.one_of_mem
  exact h

theorem mk_eqr (c : ℕ → ℕ) (a b : GM.F c) (h : GM.Rel c (GM.eqr a b)) :
  PresentedGroup.mk (GM.Rels c) a = PresentedGroup.mk (GM.Rels c) b := by
  exact (PresentedGroup.mk_eq_mk_of_mul_inv_mem h)

```

```

theorem subgroup_top_of_gens (c : ℕ → ℕ) (N : Subgroup (GM.Grp c))
  (h : ∀ g : GM.Gen c, (PresentedGroup.of g : GM.Grp c) ∈ N) : N = ⊤ := by
  have hgen : Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) : GM.Gen c → GM.Grp c) ⊆
  ↪ N.carrier := by
    rintro _ ⟨g, rfl⟩
    exact h g
  have hclosure : Subgroup.closure (Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) :
  ↪ GM.Gen c → GM.Grp c)) ≤ N :=
    ((Subgroup.closure_le (K := N)).mpr hgen)
  have htop : Subgroup.closure (Set.range (PresentedGroup.of (α := GM.Gen c) (rels := GM.Rels c) : GM.Gen c
  ↪ → GM.Grp c)) = ⊤ :=
    PresentedGroup.closure_range_of (GM.Rels c)
  rw [htop] at hclosure
  exact top_le_iff.mp hclosure

theorem cm_mem_commutator (c : ℕ → ℕ) (a b : GM.F c) :
  PresentedGroup.mk (GM.Rels c) (GM.cm a b) ∈ commutator (GM.Grp c) := by
  have h : PresentedGroup.mk (GM.Rels c) (GM.cm a b) = [[PresentedGroup.mk (GM.Rels c) a,
  PresentedGroup.mk (GM.Rels c) b]] := by
    simp [GM.cm, commutatorElement_def]
  rw [h]
  exact Subgroup.commutator_mem_commutator (Subgroup.mem_top _) (Subgroup.mem_top _)

theorem xm_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xm u) ∈ commutator (GM.Grp c) := by
  set G := GM.Grp c
  let m := PresentedGroup.mk (GM.Rels c)
  have hvalid1 : GM.Valid c 1 2 1 := by
    dsimp [GM.Valid]
    exact ⟨by decide, by decide, hc⟩
  have hvalid0 : GM.Valid c 0 2 1 := by
    dsimp [GM.Valid]
    exact ⟨by decide, by decide, hc⟩
  have hrel : GM.Rel c (GM.eqr (GM.cm (GM.xx 1 2 1 hvalid1 (u : ℂ)) (GM.yy 0 2 1 hvalid0 1)) (GM.xm ((u :
  ↪ ℂ) * 1))) :=
    GM.Rel.U3a 1 hvalid1 hvalid0 (u : ℂ) 1
  have h_eq : m (GM.cm (GM.xx 1 2 1 hvalid1 (u : ℂ)) (GM.yy 0 2 1 hvalid0 1)) = m (GM.xm ((u : ℂ) * 1)) := by
    apply mk_eqr c _ _ hrel
  have h_cm_mem : m (GM.cm (GM.xx 1 2 1 hvalid1 (u : ℂ)) (GM.yy 0 2 1 hvalid0 1)) ∈ commutator G := by
    rw [commutator_eq_closure G]
    apply Subgroup.subset_closure
    exact commutator_mem_commutatorSet (m (GM.xx 1 2 1 hvalid1 (u : ℂ))) (m (GM.yy 0 2 1 hvalid0 1))
  have h_target : m (GM.xm u) = m (GM.xm ((u : ℂ) * 1)) := by simp
  rw [h_target]
  rw [← h_eq]
  exact h_cm_mem

theorem ym_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.ym u) ∈ commutator (GM.Grp c) := by
  have hvalid0 : GM.Valid c 0 2 1 := by
    refine ⟨by decide, by decide, ?_⟩
    exact Nat.le_trans (by decide) hc
  have hvalid1 : GM.Valid c 1 2 1 := by
    refine ⟨by decide, by decide, ?_⟩
    exact Nat.le_trans (by decide) hc
  have h_rel : GM.Rel c (GM.eqr (GM.cm (GM.xx 0 2 1 hvalid0 u) (GM.yy 1 2 1 hvalid1 1))
  (GM.ym (u * 1))) :=
    GM.Rel.U3b 1 hvalid0 hvalid1 u 1

```

```

have h_eq : PresentedGroup.mk (GM.Rels c) (GM.cm (GM.xx 0 2 1 hvalid0 u) (GM.yy 1 2 1 hvalid1 1)) =
  PresentedGroup.mk (GM.Rels c) (GM.ym (u * 1)) :=
  mk_eqr c _ _ h_rel
have h_cm_mem : PresentedGroup.mk (GM.Rels c)
  (GM.cm (GM.xx 0 2 1 hvalid0 u) (GM.yy 1 2 1 hvalid1 1)) ∈ commutator (GM.Grp c) :=
  cm_mem_commutator c _ _
rw [h_eq] at h_cm_mem
simp [mul_one] using h_cm_mem

theorem wm_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.wm s) ∈ commutator (GM.Grp c) := by
have hx : PresentedGroup.mk (GM.Rels c) (GM.xm (s : ℂ)) ∈ commutator (GM.Grp c) :=
  xm_mem c hc (s : ℂ)
have hy : PresentedGroup.mk (GM.Rels c) (GM.ym (-(s : ℂ)-1)) ∈ commutator (GM.Grp c) :=
  ym_mem c hc (-(s : ℂ)-1)
have h_eq : PresentedGroup.mk (GM.Rels c) (GM.wm s) =
  PresentedGroup.mk (GM.Rels c) (GM.xm (s : ℂ)) *
  PresentedGroup.mk (GM.Rels c) (GM.ym (-(s : ℂ)-1)) *
  PresentedGroup.mk (GM.Rels c) (GM.xm (s : ℂ)) := by
  simp [GM.wm, GM.xm, GM.ym, map_mul]
rw [h_eq]
exact Subgroup.mul_mem _ (Subgroup.mul_mem _ hx hy) hx

theorem exists_unit_pow_ne_one (d : ℕ) (hd : 0 < d) :
  ∃ s : ℂ×, (s : ℂ) ^ d ≠ 1 := by
have h2val : (2 : ℂ) ^ d ≠ 1 := by
  intro h
  have hnormSq : Complex.normSq ((2 : ℂ) ^ d) = Complex.normSq (1 : ℂ) := by rw [h]
  have h_one : Complex.normSq (1 : ℂ) = 1 := by simp
  rw [h_one] at hnormSq
  have hcalc : Complex.normSq ((2 : ℂ) ^ d) = (Complex.normSq (2 : ℂ)) ^ d := by
    simp using map_pow Complex.normSq (2 : ℂ) d
  have h4 : Complex.normSq (2 : ℂ) = (4 : ℝ) := by norm_num
  rw [hcalc, h4] at hnormSq

  have h4pos : (0 : ℝ) < 4 := by norm_num
  have h4dpos : 0 < (4 : ℝ) ^ d := pow_pos h4pos d
  have h4dgt1 : (1 : ℝ) < (4 : ℝ) ^ d := by
    have h_nat : 1 < (4 : ℕ) ^ d := Nat.one_lt_pow (by omega : d ≠ 0) (by norm_num : 1 < 4)
    exact_mod_cast h_nat
  linarith
  refine ⟨Units.mk0 (2 : ℂ) (by norm_num), h2val⟩

theorem xx_add_mk (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (u + v)) =
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u) *
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h v) := by
have hrel : GM.Rel c (GM.eqr (GM.xx l j k h u * GM.xx l j k h v) (GM.xx l j k h (u + v))) :=
  GM.Rel.Im1a l j k h u v
have h_eq := mk_eqr c (GM.xx l j k h u * GM.xx l j k h v) (GM.xx l j k h (u + v)) hrel
calc
  PresentedGroup.mk (GM.Rels c) (GM.xx l j k h (u + v))
    = PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u * GM.xx l j k h v) := by
  rw [← h_eq]
  _ = PresentedGroup.mk (GM.Rels c) (GM.xx l j k h u) *
    PresentedGroup.mk (GM.Rels c) (GM.xx l j k h v) := by
  rw [map_mul]

theorem yy_add_mk (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u v : ℂ) :

```

```

PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (u + v)) =
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h u) *
    PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) := by
have hrel : GM.Rel c (GM.eqr (GM.yy 1 j k h u * GM.yy 1 j k h v) (GM.yy 1 j k h (u + v))) :=
  GM.Rel.Im1b 1 j k h u v
have := mk_eqr c (GM.yy 1 j k h u * GM.yy 1 j k h v) (GM.yy 1 j k h (u + v)) hrel
simp [GM.eqr, mul_inv_rev] using this.symm

theorem xx_sub_mk (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (a b : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (a - b)) =
    PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h a) *
      (PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h b))-1 := by
have hzero : PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (0 : ℂ)) = 1 := by
  have hrel : GM.Rel c (GM.eqr (GM.xx 1 j k h (0 : ℂ)) 1) := by
    have := GM.Rel.Im1a 1 j k h (0 : ℂ) (0 : ℂ)
  simp [GM.eqr] using this
  have hmem : GM.eqr (GM.xx 1 j k h (0 : ℂ)) 1 ∈ GM.Rels c := hrel
  exact PresentedGroup.one_of_mem hmem
have hneg (u : ℂ) : PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (-u)) =
  (PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h u))-1 := by
have h := xx_add_mk c l j k h u (-u)

have hzero' : (u + (-u : ℂ)) = (0 : ℂ) := by ring
rw [hzero'] at h
rw [hzero] at h

exact eq_inv_of_mul_eq_one_right h.symm
calc
  PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (a - b))
    = PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (a + (-b))) := by ring
_ = PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h a) *
  PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (-b)) := xx_add_mk c l j k h a (-b)
_ = PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h a) *
  (PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h b))-1 := by rw [hneg b]

theorem yy_sub_mk (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (a b : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b)) =
    PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h a) *
      (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b))-1 := by
have hRel : GM.Rel c (GM.eqr (GM.yy 1 j k h (a - b) * GM.yy 1 j k h b) (GM.yy 1 j k h a)) := by
  have := GM.Rel.Im1b 1 j k h (a - b) b
  simp [sub_add_cancel] using this
have hEq := mk_eqr c (GM.yy 1 j k h (a - b) * GM.yy 1 j k h b) (GM.yy 1 j k h a) hRel

have hMul : PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b) * GM.yy 1 j k h b) =
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b)) *
    PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b) := by
  simp
  rw [hMul] at hEq

calc
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b))
    = (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (a - b)) *
      PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b)) *
      (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b))-1 := by

```

```

simp
_ = PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h a) *
  (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h b))-1 := by
  rw [hEq]

theorem xx_conj_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) (v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h ((s : ℂ) ^ (j - 1) * v)) *
    (PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h v))-1 ∈ commutator (GM.Grp c) := by
  have him6b := GM.Rel.Im6b 1 j k h s v
  have hmem : GM.eqr (GM.h2 s * GM.xx 1 j k h v * (GM.h2 s)-1) (GM.xx 1 j k h ((s : ℂ) ^ (j - 1) * v)) ∈
    ↪ GM.Rels c := him6b
  have heq := PresentedGroup.mk_eq_mk_of_mul_inv_mem hmem
  rw [← heq]
  simp [map_mul, map_inv, commutatorElement_def] using Subgroup.commutator_mem_commutator
  (g1 := PresentedGroup.mk (GM.Rels c) (GM.h2 s))
  (g2 := PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h v))
  (h1 := show PresentedGroup.mk (GM.Rels c) (GM.h2 s) ∈ (T : Subgroup (GM.Grp c)) from Subgroup.mem_top _)
  (h2 := show PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h v) ∈ (T : Subgroup (GM.Grp c)) from
    ↪ Subgroup.mem_top _)

open GM in
theorem yy_conj_mem_aiprover_s0 (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) (v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v))-1 ∈ commutator (GM.Grp c) := by
  have h_rel := Rel.Im6d 1 j k h s v
  have h_eq := mk_eqr c (GM.h2 s * GM.yy 1 j k h v * (GM.h2 s)-1)
    (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) h_rel
  have h_mk : PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) =
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c)-1 := by
    calc
    PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) =
      PresentedGroup.mk (GM.Rels c) (GM.h2 s * GM.yy 1 j k h v * (GM.h2 s)-1) := h_eq.symm
    _ = (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c) *
      (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c) *
      (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c)-1 := by simp [map_mul]
  rw [h_mk]
  have h_comm : (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c)-1 *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c)-1 =
    [[(PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c),
      (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c)]] := rfl
  rw [h_comm]
  dsimp [commutator]
  apply Subgroup.subset_closure
  exact ⟨(PresentedGroup.mk (GM.Rels c) (GM.h2 s) : GM.Grp c), Subgroup.mem_top _,
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v) : GM.Grp c), Subgroup.mem_top _, rfl⟩

theorem yy_conj_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) (v : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1 * v)) *
    (PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h v))-1 ∈ commutator (GM.Grp c) := by
  first
  | exact yy_conj_mem_aiprover_s0
  | (intros; apply yy_conj_mem_aiprover_s0 <;> assumption)

theorem xx_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h u) ∈ commutator (GM.Grp c) := by

```

```

have hpos_or : j - 1 = 0 ∨ 0 < j - 1 := Nat.eq_zero_or_pos (j - 1)
rcases hpos_or with (hjl | hdpos)
· rcases h with ⟨hlt, _, _⟩
  have : 1 < 1 :=
    Nat.lt_of_lt_of_le hlt (Nat.le_of_sub_eq_zero hjl)
  exact absurd this (Nat.lt_irrefl _)
· rcases exists_unit_pow_ne_one (j - 1) hdpos with ⟨s, hs⟩
  have hden_ne_zero : (s : ℂ) ^ (j - 1) - 1 ≠ 0 := by
    intro hzero
    have h_one : (s : ℂ) ^ (j - 1) = 1 := sub_eq_zero.mp hzero
    exact hs h_one
  set v := u / ((s : ℂ) ^ (j - 1) - 1) with hv_def
  have h_eq : u = ((s : ℂ) ^ (j - 1) - 1) * v := by
    dsimp [v]
    field_simp [hden_ne_zero]
  rw [h_eq]
  have : ((s : ℂ) ^ (j - 1) - 1) * v = ((s : ℂ) ^ (j - 1)) * v - v := by ring
  rw [this]
  rw [xx_sub_mk c 1 j k h ((s : ℂ) ^ (j - 1)) * v]
  exact xx_conj_mem c 1 j k h s v

theorem yy_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c 1 j k) (u : ℂ) :
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h u) ∈ commutator (GM.Grp c) := by
  have hjl : 0 < j - 1 := by
    have := h.1
    omega
  obtain ⟨s, hs⟩ := exists_unit_pow_ne_one (j - 1) hjl
  have ht : ((s : ℂ) ^ (j - 1))-1 - 1 ≠ 0 := by
    intro h0
    apply hs
  have h1 : ((s : ℂ) ^ (j - 1))-1 = 1 := sub_eq_zero.mp h0
  first
    | exact inv_eq_one.mp h1
    | simpa using h1
  have hu : ((s : ℂ) ^ (j - 1))-1 * (u / ((s : ℂ) ^ (j - 1))-1 - 1) -
    u / ((s : ℂ) ^ (j - 1))-1 - 1 = u := by
    rw [← sub_one_mul]
  first
    | exact mul_div_cancel0 u ht
    | (rw [mul_comm]; exact div_mul_cancel0 u ht)
    | field_simp
  have key := yy_conj_mem c 1 j k h s (u / ((s : ℂ) ^ (j - 1))-1 - 1)
  first
    | (rw [← yy_sub_mk c 1 j k h, hu] at key; exact key)
    | (rw [← hu, yy_sub_mk c 1 j k h]; exact key)

theorem ww_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c 1 j k) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.ww 1 j k h s) ∈ commutator (GM.Grp c) := by
  let f : GM.F c →* GM.Grp c := PresentedGroup.mk (GM.Rels c)
  have hxx : f (GM.xx 1 j k h (s : ℂ)) ∈ commutator (GM.Grp c) := by
    dsimp [f]
    simpa using xx_mem c 1 j k h (s : ℂ)
  have hyy : f (GM.yy 1 j k h (-(s : ℂ)-1 / GM.cc 1 j)) ∈ commutator (GM.Grp c) := by
    simpa using yy_mem c 1 j k h (-(s : ℂ)-1 / GM.cc 1 j)

  have hprod : f (GM.xx 1 j k h (s : ℂ) * GM.yy 1 j k h (-(s : ℂ)-1 / GM.cc 1 j) * GM.xx 1 j k h (s : ℂ))
  → ∈ commutator (GM.Grp c) := by
    have hcalc : f (GM.xx 1 j k h (s : ℂ) * GM.yy 1 j k h (-(s : ℂ)-1 / GM.cc 1 j) * GM.xx 1 j k h (s :
  → ℂ)) =

```

```

      f (GM.xx 1 j k h (s : ℂ)) * f (GM.yy 1 j k h (-(s : ℂ)-1 / GM.cc 1 j)) * f (GM.xx 1 j k h (s :
↪ ℂ)) := by
  simp [map_mul]
  rw [hcalc]
  exact Subgroup.mul_mem _ (Subgroup.mul_mem _ hxx hyy) hxx

  have hww : GM.wv 1 j k h s = GM.xx 1 j k h (s : ℂ) * GM.yy 1 j k h (-(s : ℂ)-1 / GM.cc 1 j) * GM.xx 1 j
↪ k h (s : ℂ) := rfl
  rw [hww]
  exact hprod

theorem h1h2_neg_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 (-s) * GM.h2 (-(s-1))) ∈ commutator (GM.Grp c) := by
  have hrel : GM.Rel c (GM.eqr (GM.wm s * GM.wm 1) (GM.h1 (-s) * GM.h2 (-(s-1)))) :=
    GM.Rel.Re3 s
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.wm s * GM.wm 1) =
    PresentedGroup.mk (GM.Rels c) (GM.h1 (-s) * GM.h2 (-(s-1))) :=
    mk_eqr c (GM.wm s * GM.wm 1) (GM.h1 (-s) * GM.h2 (-(s-1))) hrel
  have h_wm_s : PresentedGroup.mk (GM.Rels c) (GM.wm s) ∈ commutator (GM.Grp c) :=
    wm_mem c hc s
  have h_wm_1 : PresentedGroup.mk (GM.Rels c) (GM.wm 1) ∈ commutator (GM.Grp c) :=
    wm_mem c hc 1
  have h_prod : PresentedGroup.mk (GM.Rels c) (GM.wm s * GM.wm 1) ∈ commutator (GM.Grp c) :=
    Subgroup.mul_mem _ h_wm_s h_wm_1
  rw [← h_eq]
  exact h_prod

theorem h1h2_inv_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (x : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 x * GM.h2 x-1) ∈ commutator (GM.Grp c) := by
  have hrel := GM.Rel.Re3 (c := c) (-x)
  have h_simplified : (GM.h1 (c := c) (-(x)) * GM.h2 (c := c) (-(x-1))) = (GM.h1 (c := c) x * GM.h2 (c
↪ := c) x-1) := by
  simp
  have hmem : PresentedGroup.mk (GM.Rels c) (GM.wm (-x) * GM.wm 1) ∈ commutator (GM.Grp c) := by
  have h1 : PresentedGroup.mk (GM.Rels c) (GM.wm (-x)) ∈ commutator (GM.Grp c) := wm_mem c hc (-x)
  have h2 : PresentedGroup.mk (GM.Rels c) (GM.wm 1) ∈ commutator (GM.Grp c) := wm_mem c hc 1
  exact Subgroup.mul_mem _ h1 h2
  have heq := mk_eqr c (GM.wm (-x) * GM.wm 1) (GM.h1 x * GM.h2 x-1) (by
    simpa [h_simplified] using hrel)
  rw [← heq]
  exact hmem

theorem im3_mem (c : ℕ → ℕ) (l j k : ℕ) (h : GM.Valid c l j k) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (l + 1)))
  ∈ commutator (GM.Grp c) := by
  set w1 := GM.wv 1 j k h (s ^ ((l + 1) * (j - 1))) with hw1
  set w0 := GM.wv 1 j k h 1 with hw0
  have hIm3_rel : GM.Rel c (GM.eqr (w1 * w0) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (l + 1)))) :=
    GM.Rel.Im3 l j k h s
  have h_mul_inv : w1 * w0 * (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (l + 1)))-1 ∈ GM.Rels c := by
    simpa [GM.Rels] using hIm3_rel
  have h_mem' : PresentedGroup.mk (GM.Rels c) (GM.eqr (w1 * w0) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (l +
↪ 1)))) = 1 :=
    PresentedGroup.one_of_mem hIm3_rel
  have h_eq : PresentedGroup.mk (GM.Rels c) (w1 * w0) =
    PresentedGroup.mk (GM.Rels c) (GM.h1 ((-s) ^ (j - 1)) * GM.h2 ((-s) ^ (l + 1))) := by
    apply PresentedGroup.mk_eq_mk_of_mul_inv_mem h_mul_inv
  have h_prod_mem : PresentedGroup.mk (GM.Rels c) (w1 * w0) ∈ commutator (GM.Grp c) := by
  have h1 : PresentedGroup.mk (GM.Rels c) w1 ∈ commutator (GM.Grp c) :=

```

```

ww_mem c 1 j k h (s ^ ((1 + 1) * (j - 1)))
have h2 : PresentedGroup.mk (GM.Rels c) w0 ∈ commutator (GM.Grp c) :=
  ww_mem c 1 j k h 1
exact Subgroup.mul_mem (commutator (GM.Grp c)) h1 h2
rw [← h_eq]
exact h_prod_mem

```

```

theorem h1sq_h2_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (y : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 (y ^ 2) * GM.h2 y) ∈ commutator (GM.Grp c) := by
  have hvalid : GM.Valid c 0 2 1 := ⟨by decide, by decide, hc⟩
  have hw1 := ww_mem c 0 2 1 hvalid (y ^ 2)
  have hw2 := ww_mem c 0 2 1 hvalid 1
  have hprod : PresentedGroup.mk (GM.Rels c)
    (GM.ww 0 2 1 hvalid (y ^ 2) * GM.ww 0 2 1 hvalid 1) ∈ commutator (GM.Grp c) :=
    (commutator (GM.Grp c)).mul_mem hw1 hw2
  have him3 : GM.Rel c (GM.eqr (GM.ww 0 2 1 hvalid (y ^ 2) * GM.ww 0 2 1 hvalid 1)
    (GM.h1 (y ^ 2) * GM.h2 y)) := by
    have := GM.Rel.Im3 0 2 1 hvalid (-y)
    simp [show ((-y : ℂ×) : ℂ) ^ ((0+1)*(2-0)) = (y : ℂ)^2 by norm_num] using this
  have heq : PresentedGroup.mk (GM.Rels c)
    (GM.ww 0 2 1 hvalid (y ^ 2) * GM.ww 0 2 1 hvalid 1) =
    PresentedGroup.mk (GM.Rels c) (GM.h1 (y ^ 2) * GM.h2 y) := by
    simp [GM.Rels] using PresentedGroup.mk_eq_mk_of_mul_inv_mem him3
  rw [← heq]
  exact hprod

```

```

theorem h1_cube_eq (c : ℕ → ℕ) (z : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 (z ^ 3)) =
  PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 z-1) *
  (PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1) ^ 2) * GM.h2 z-1))-1 := by
  let φ := PresentedGroup.mk (GM.Rels c)
  have hφ_mul : ∀ x y : GM.F c, φ (x * y) = φ x * φ y := by
    intro x y; exact MonoidHom.map_mul φ x y
  have hφ_inv : ∀ x : GM.F c, φ (x-1) = (φ x)-1 := by
    intro x; exact MonoidHom.map_inv φ x

```

```

have h1_mul : ∀ s t : ℂ×, φ (GM.h1 (c := c) (s * t)) = φ (GM.h1 (c := c) s) * φ (GM.h1 (c := c) t) := by
  intro s t
  have hRel : GM.Rel c (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) t) (GM.h1 (c := c) (s * t))) :=
    GM.Rel.H1a (c := c) s t
  have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)

```

```

have hcalc : φ (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) t) (GM.h1 (c := c) (s * t))) =
  φ (GM.h1 (c := c) s * GM.h1 (c := c) t) * (φ (GM.h1 (c := c) (s * t)))-1 := by
  simp [GM.eqr, hφ_mul, hφ_inv]
rw [hcalc] at h_one

```

```

have h_eq : φ (GM.h1 (c := c) s * GM.h1 (c := c) t) = φ (GM.h1 (c := c) (s * t)) := by
  calc
  φ (GM.h1 (c := c) s * GM.h1 (c := c) t) =
    (φ (GM.h1 (c := c) s * GM.h1 (c := c) t) * (φ (GM.h1 (c := c) (s * t)))-1) *
    φ (GM.h1 (c := c) (s * t)) := by group

```

```

    _ = 1 *  $\varphi$  (GM.h1 (c := c) (s * t)) := by rw [h_one]
    _ =  $\varphi$  (GM.h1 (c := c) (s * t)) := by simp
  rw [h $\varphi$ _mul] at h_eq
  exact h_eq.symm

have h1_one :  $\varphi$  (GM.h1 (c := c) 1) = 1 := by
  have hRel : GM.Rel c (GM.eqr (GM.h1 (c := c) 1 * GM.h1 (c := c) 1) (GM.h1 (c := c) (1 * 1))) :=
    GM.Rel.H1a (c := c) 1 1
  have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)

have hcalc :  $\varphi$  (GM.eqr (GM.h1 (c := c) 1 * GM.h1 (c := c) 1) (GM.h1 (c := c) (1 * 1))) =
   $\varphi$  (GM.h1 (c := c) 1 * GM.h1 (c := c) 1) * ( $\varphi$  (GM.h1 (c := c) (1 * 1)))-1 := by
  simp [GM.eqr, h $\varphi$ _mul]
  rw [hcalc] at h_one

rw [mul_one] at h_one

rw [h $\varphi$ _mul] at h_one

calc
   $\varphi$  (GM.h1 (c := c) 1) = ( $\varphi$  (GM.h1 (c := c) 1) *  $\varphi$  (GM.h1 (c := c) 1)) * ( $\varphi$  (GM.h1 (c := c) 1))-1 := by
  ↪ group
  _ = 1 := h_one

have h1_inv :  $\forall s : \mathbb{C}^\times, \varphi$  (GM.h1 (c := c) s-1) = ( $\varphi$  (GM.h1 (c := c) s))-1 := by
  intro s
  have hRel : GM.Rel c (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) s-1) (GM.h1 (c := c) (s * s-1))) :=
    GM.Rel.H1a (c := c) s s-1

have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)
have hcalc :  $\varphi$  (GM.eqr (GM.h1 (c := c) s * GM.h1 (c := c) s-1) (GM.h1 (c := c) (s * s-1))) =
   $\varphi$  (GM.h1 (c := c) s * GM.h1 (c := c) s-1) * ( $\varphi$  (GM.h1 (c := c) (s * s-1)))-1 := by
  simp [GM.eqr, h $\varphi$ _mul, h $\varphi$ _inv]
  rw [hcalc] at h_one

have h_s_one :  $\varphi$  (GM.h1 (c := c) (s * s-1)) = 1 := by
  have : s * s-1 = (1 :  $\mathbb{C}^\times$ ) := by simp
  rw [this]
  exact h1_one
  rw [h_s_one] at h_one

have : (1 : PresentedGroup (GM.Rels c))-1 = 1 := by simp
  rw [this, mul_one] at h_one

calc
   $\varphi$  (GM.h1 (c := c) s-1) = 1 *  $\varphi$  (GM.h1 (c := c) s-1) := by simp
  _ = (( $\varphi$  (GM.h1 (c := c) s))-1 *  $\varphi$  (GM.h1 (c := c) s)) *  $\varphi$  (GM.h1 (c := c) s-1) := by simp
  _ = ( $\varphi$  (GM.h1 (c := c) s))-1 * ( $\varphi$  (GM.h1 (c := c) s) *  $\varphi$  (GM.h1 (c := c) s-1)) := by group
  _ = ( $\varphi$  (GM.h1 (c := c) s))-1 * 1 := by
  rw [show  $\varphi$  (GM.h1 (c := c) s) *  $\varphi$  (GM.h1 (c := c) s-1) = 1 from h_one]

```

```

_ = (φ (GM.h1 (c := c) s))-1 := by simp

have h_pow_h1 : ∀ (x : ℂ×) (n : ℕ), φ (GM.h1 (c := c) (x ^ n)) = (φ (GM.h1 (c := c) x)) ^ n := by
  intro x n
  induction' n with k ih
  ·
    simp [h1_one]
  ·
    rw [pow_succ, pow_succ]

    rw [h1_mul (x ^ k) x]
    rw [ih]

have h_inv_h1 : ∀ (x : ℂ×), φ (GM.h1 (c := c) x-1) = (φ (GM.h1 (c := c) x))-1 := h1_inv

calc
  φ (GM.h1 (c := c) (z ^ 3)) = (φ (GM.h1 (c := c) z)) ^ 3 := by rw [h_pow_h1]
  _ = (φ (GM.h1 (c := c) z)) * ((φ (GM.h1 (c := c) z))-1 ^ 2)-1 := by group
  _ = (φ (GM.h1 (c := c) z) * φ (GM.h2 (c := c) z-1)) * (((φ (GM.h1 (c := c) z))-1 ^ 2) * φ (GM.h2 (c :=
→ c) z-1))-1 := by group
  _ = (φ (GM.h1 (c := c) z) * φ (GM.h2 (c := c) z-1)) * ((φ (GM.h1 (c := c) (z-1 ^ 2)) * φ (GM.h2 (c :=
→ c) z-1))-1) := by
    simp [h_pow_h1, h_inv_h1]
  _ = PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 z-1) *
    (PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1 ^ 2) * GM.h2 z-1))-1 := by
    simp [φ, hφ_mul]

theorem exists_unit_cube (s : ℂ×) : ∃ z : ℂ×, z ^ 3 = s := by
  obtain ⟨w, hw⟩ := IsAlgClosed.exists_pow_nat_eq (s : ℂ) (by decide : 0 < 3)
  have hw_ne_zero : w ≠ 0 := by
    intro hzero
    rw [hzero, zero_pow (by decide : 3 ≠ 0)] at hw
  have hs_ne_zero : (s : ℂ) ≠ 0 := Units.ne_zero s
  exact hs_ne_zero (Eq.symm hw)
  let z : ℂ× := Units.mk0 w hw_ne_zero
  refine ⟨z, ?_⟩
  apply Units.ext_iff.mpr
  calc
    (z : ℂ) ^ 3 = w ^ 3 := rfl
    _ = (s : ℂ) := hw

theorem h1_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 s) ∈ commutator (GM.Grp c) := by
  rcases exists_unit_cube s with ⟨z, hz⟩

  have h_comm_z : PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 (z-1)) ∈ commutator (GM.Grp c) :=
    h1h2_inv_mem c hc z

  have h_comm_z_inv : PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1 ^ 2) * GM.h2 (z-1)) ∈
    commutator (GM.Grp c) :=
    h1sq_h2_mem c hc (z-1)

```

```

have h_cube := h1_cube_eq c z

have h_prod : PresentedGroup.mk (GM.Rels c) (GM.h1 z * GM.h2 (z-1)) *
  (PresentedGroup.mk (GM.Rels c) (GM.h1 ((z-1)2 * GM.h2 (z-1)))-1 ∈
  commutator (GM.Grp c) := by
  apply Subgroup.mul_mem
  · exact h_comm_z
  · apply Subgroup.inv_mem
    exact h_comm_z_inv

have h_cube_mem : PresentedGroup.mk (GM.Rels c) (GM.h1 (z3)) ∈ commutator (GM.Grp c) := by
  rw [h_cube]
  exact h_prod

simp [hz] using h_cube_mem

theorem h2_eq_conj (c : ℕ → ℕ) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s) =
    PresentedGroup.mk (GM.Rels c) (GM.wm 1) * PresentedGroup.mk (GM.Rels c) (GM.h1 s) *
      (PresentedGroup.mk (GM.Rels c) (GM.wm 1))-1 := by

have h_eq := mk_eqr c (GM.wm 1 * GM.h1 s * (GM.wm 1)-1) (GM.h2 s) (by
  apply GM.Re1.Re5a)

simp [PresentedGroup.mk, map_mul, map_inv] using h_eq.symm

theorem h2_mem (c : ℕ → ℕ) (hc : 1 ≤ c 2) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s) ∈ commutator (GM.Grp c) := by
  have hnormal : (commutator (GM.Grp c)).Normal := inferInstance
  have hh1 : PresentedGroup.mk (GM.Rels c) (GM.h1 s) ∈ commutator (GM.Grp c) := h1_mem c hc s
  have hww : PresentedGroup.mk (GM.Rels c) (GM.wm 1) ∈ commutator (GM.Grp c) := wm_mem c hc 1
  rw [h2_eq_conj c s]
  exact hnormal.conj_mem (PresentedGroup.mk (GM.Rels c) (GM.h1 s)) hh1
    (PresentedGroup.mk (GM.Rels c) (GM.wm 1))

theorem gsimple_g_m_perfect (c : ℕ → ℕ) (hc : 1 ≤ c 2) :
  commutator (GM.Grp c) = T := by
  apply subgroup_top_of_gens
  intro g
  cases g with
  | H1 s => exact h1_mem c hc s
  | H2 s => exact h2_mem c hc s
  | Xm u => exact xm_mem c hc u
  | Ym u => exact ym_mem c hc u
  | X l j k h u => exact xx_mem c l j k h u
  | Y l j k h u => exact yy_mem c l j k h u

namespace GM

/-- Generators of the toral subgroup: the images of H_1(s) and H_2(s) in G(m). -/
def torusGens (c : ℕ → ℕ) : Set (Grp c) :=
  {g | ∃ s : ℂ×, g = PresentedGroup.mk (Rels c) (h1 s) ∨
    g = PresentedGroup.mk (Rels c) (h2 s)}

/-- The toral subgroup H of G(m). -/

```

```

def torusSub (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) : Subgroup (Grp c) := Subgroup.closure (torusGens c)

/-- The unipotent generators  $X_{\{-1\}}(u)$ ,  $Y_{\{-1\}}(u)$ ,  $X_{\{\ell,jk\}}(u)$ ,  $Y_{\{\ell,jk\}}(u)$ , viewed in  $G(m)$ . -/
def unipGens (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) : Set (Grp c) :=
  {g | ( $\exists u : \mathbb{C}$ , g = PresentedGroup.mk (Rels c) (xm u))  $\vee$ 
    ( $\exists u : \mathbb{C}$ , g = PresentedGroup.mk (Rels c) (ym u))  $\vee$ 
    ( $\exists (l j k : \mathbb{N})$  (h : Valid c l j k) (u :  $\mathbb{C}$ ),
      g = PresentedGroup.mk (Rels c) (xx l j k h u))  $\vee$ 
    ( $\exists (l j k : \mathbb{N})$  (h : Valid c l j k) (u :  $\mathbb{C}$ ),
      g = PresentedGroup.mk (Rels c) (yy l j k h u))}

/-- The unipotent subgroup U of  $G(m)$ . -/
def unipSub (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) : Subgroup (Grp c) := Subgroup.closure (unipGens c)

end GM

theorem h1_conj_unip (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) (s :  $\mathbb{C}^\times$ ) (g : GM.Grp c) (hg : g  $\in$  GM.unipGens c) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 s) * g * (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1
   $\in$  GM.unipGens c := by

rcases hg with (<u, hg_u> | <u, hg_u> | <l, j, k, h, u, hg_u> | <l, j, k, h, u, hg_u>)
.
  subst hg_u

  have hrel := GM.Rel.Re6a (c := c) s u
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.h1 s * GM.xm u * (GM.h1 s)-1) =
    PresentedGroup.mk (GM.Rels c) (GM.xm ((s :  $\mathbb{C}$ ) * u)) := by
    simpa [GM.eqr] using mk_eqr c (GM.h1 s * GM.xm u * (GM.h1 s)-1) (GM.xm ((s :  $\mathbb{C}$ ) * u)) hrel
  have h_target : PresentedGroup.mk (GM.Rels c) (GM.xm ((s :  $\mathbb{C}$ ) * u))  $\in$  GM.unipGens c := by
    refine Or.inl (<(s :  $\mathbb{C}$ ) * u, ?_>)
    rfl
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h1 s) * PresentedGroup.mk (GM.Rels c) (GM.xm u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.xm ((s :  $\mathbb{C}$ ) * u)) := by
    simpa [GM.xm, map_mul, map_inv] using h_eq
  rw [h_eq']
  exact h_target
.

  subst hg_u

  have hrel := GM.Rel.Re6c (c := c) s u
  have h_eq : PresentedGroup.mk (GM.Rels c) (GM.h1 s * GM.ym u * (GM.h1 s)-1) =
    PresentedGroup.mk (GM.Rels c) (GM.ym (((s :  $\mathbb{C}$ )-1) * u)) := by
    simpa [GM.eqr] using mk_eqr c (GM.h1 s * GM.ym u * (GM.h1 s)-1) (GM.ym (((s :  $\mathbb{C}$ )-1) * u)) hrel
  have h_target : PresentedGroup.mk (GM.Rels c) (GM.ym (((s :  $\mathbb{C}$ )-1) * u))  $\in$  GM.unipGens c := by
    refine Or.inr (Or.inl (<((s :  $\mathbb{C}$ )-1) * u, ?_>))
    rfl
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h1 s) * PresentedGroup.mk (GM.Rels c) (GM.ym u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.ym (((s :  $\mathbb{C}$ )-1) * u))
   $\hookrightarrow$  := by
    simpa [GM.ym, map_mul, map_inv] using h_eq
  rw [h_eq']
  exact h_target
.

  subst hg_u

  have hrel := GM.Rel.Im6a (c := c) l j k h s u

```

```

have h_eq : PresentedGroup.mk (GM.Rels c) (GM.h1 s * GM.xx 1 j k h u * (GM.h1 s)-1) =
  PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h ((s : ℂ) ^ (1 + 1) * u)) := by
  simpa [GM.eqr] using mk_eqr c (GM.h1 s * GM.xx 1 j k h u * (GM.h1 s)-1) (GM.xx 1 j k h ((s : ℂ) ^ (1 +
→ 1) * u)) hrel
have h_target : PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h ((s : ℂ) ^ (1 + 1) * u)) ∈ GM.unipGens c
→ := by
  refine Or.inr (Or.inr (Or.inl ⟨1, j, k, h, ((s : ℂ) ^ (1 + 1) * u), ?_⟩))
  rfl
have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h1 s) * PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h u) *
  (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h ((s : ℂ) ^
→ (1 + 1) * u)) := by
  simpa [GM.xx, map_mul, map_inv] using h_eq
  rw [h_eq']
  exact h_target
.
subst hg_u

have hrel := GM.Rel.Im6c (c := c) 1 j k h s u
have h_eq : PresentedGroup.mk (GM.Rels c) (GM.h1 s * GM.yy 1 j k h u * (GM.h1 s)-1) =
  PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (1 + 1))-1 * u)) := by
  simpa [GM.eqr] using mk_eqr c (GM.h1 s * GM.yy 1 j k h u * (GM.h1 s)-1) (GM.yy 1 j k h (((s : ℂ) ^ (1
→ + 1))-1 * u)) hrel
have h_target : PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ) ^ (1 + 1))-1 * u)) ∈ GM.unipGens
→ c := by
  refine Or.inr (Or.inr (Or.inr ⟨1, j, k, h, (((s : ℂ) ^ (1 + 1))-1 * u), ?_⟩))
  rfl
have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h1 s) * PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h u) *
  (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ)
→ ^ (1 + 1))-1 * u)) := by
  simpa [GM.yy, map_mul, map_inv] using h_eq
  rw [h_eq']
  exact h_target

theorem h2_conj_unip (c : ℕ → ℕ) (s : ℂ×) (g : GM.Grp c) (hg : g ∈ GM.unipGens c) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s) * g * (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1
  ∈ GM.unipGens c := by
  rcases hg with ⟨u, hg'⟩ | ⟨u, hg'⟩ | ⟨1, j, k, h, u, hg'⟩ | ⟨1, j, k, h, u, hg'⟩
  .
  subst hg'
  have h_rel := GM.Rel.Re6b (c := c) s u
  have h_eq := mk_eqr c (GM.h2 s * GM.xm u * (GM.h2 s)-1) (GM.xm ((s : ℂ)-1 * u)) h_rel
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h2 s) * PresentedGroup.mk (GM.Rels c) (GM.xm u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.xm ((s : ℂ)-1 * u))
  → := by
    simpa [PresentedGroup.mk, map_mul, map_inv] using h_eq
    rw [h_eq']
    exact Set.mem_setOf.mpr (Or.inl ⟨(s : ℂ)-1 * u, rfl⟩)
  .
  subst hg'
  have h_rel := GM.Rel.Re6d (c := c) s u
  have h_eq := mk_eqr c (GM.h2 s * GM.ym u * (GM.h2 s)-1) (GM.ym ((s : ℂ) * u)) h_rel
  have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h2 s) * PresentedGroup.mk (GM.Rels c) (GM.ym u) *
    (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.ym ((s : ℂ) * u)) := by
    simpa [PresentedGroup.mk, map_mul, map_inv] using h_eq
    rw [h_eq']
    exact Set.mem_setOf.mpr (Or.inr (Or.inl ⟨(s : ℂ) * u, rfl⟩))
  .
  subst hg'
  have h_rel := GM.Rel.Im6b (c := c) 1 j k h s u

```

```

    have h_eq := mk_eqr c (GM.h2 s * GM.xx 1 j k h u * (GM.h2 s)-1) (GM.xx 1 j k h (((s : ℂ) ^ (j - 1)) * u)) h_rel
  ↪
    have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h2 s) * PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h u) *
      (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.xx 1 j k h (((s : ℂ)
  ↪
    ^ (j - 1)) * u)) := by
      simpa [PresentedGroup.mk, map_mul, map_inv] using h_eq
      rw [h_eq']
      exact Set.mem_setOf.mpr (Or.inr (Or.inr (Or.inl ⟨1, j, k, h, ((s : ℂ) ^ (j - 1)) * u, rfl⟩)))
    .
    subst hg'
    have h_rel := GM.Rel.Im6d (c := c) 1 j k h s u
    have h_eq := mk_eqr c (GM.h2 s * GM.yy 1 j k h u * (GM.h2 s)-1) (GM.yy 1 j k h (((s : ℂ) ^ (j - 1))-1
  ↪
    * u)) h_rel
    have h_eq' : PresentedGroup.mk (GM.Rels c) (GM.h2 s) * PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h u) *
      (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 = PresentedGroup.mk (GM.Rels c) (GM.yy 1 j k h (((s : ℂ)
  ↪
    ^ (j - 1))-1 * u)) := by
      simpa [PresentedGroup.mk, map_mul, map_inv] using h_eq
      rw [h_eq']
      exact Set.mem_setOf.mpr (Or.inr (Or.inr (Or.inr ⟨1, j, k, h, (((s : ℂ) ^ (j - 1))-1 * u, rfl⟩)))

theorem h1_inv_mk (c : ℕ → ℕ) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h1 s-1) = (PresentedGroup.mk (GM.Rels c) (GM.h1 s))-1 := by
  let φ := PresentedGroup.mk (GM.Rels c)
  have hφ_mul : ∀ x y : GM.F c, φ (x * y) = φ x * φ y := by
    intro x y; exact MonoidHom.map_mul φ x y
  have hφ_inv : ∀ x : GM.F c, φ (x-1) = (φ x)-1 := by
    intro x; exact MonoidHom.map_inv φ x
  have h1_one : φ (GM.h1 1) = 1 := by
    have hRel : GM.Rel c (GM.eqr (GM.h1 1 * GM.h1 1) (GM.h1 (1 * 1))) :=
      GM.Rel.H1a (c := c) 1 1
    have h_one := PresentedGroup.one_of_mem (hx := hRel) (rels := GM.Rels c)
    have hcalc : φ (GM.eqr (GM.h1 1 * GM.h1 1) (GM.h1 (1 * 1))) =
      φ (GM.h1 1 * GM.h1 1) * (φ (GM.h1 (1 * 1)))-1 := by
        simp [GM.eqr, hφ_mul, hφ_inv]
      rw [hcalc] at h_one
      rw [mul_one] at h_one
      rw [hφ_mul] at h_one
    calc
      φ (GM.h1 1) = (φ (GM.h1 1) * φ (GM.h1 1)) * (φ (GM.h1 1))-1 := by group
      _ = 1 := h_one
  have hRel : GM.Rel c (GM.eqr (GM.h1 s * GM.h1 s-1) (GM.h1 (s * s-1))) :=
    GM.Rel.H1a (c := c) s s-1
  have hRel' : GM.Rel c (GM.eqr (GM.h1 s * GM.h1 s-1) (GM.h1 1)) := by
    simpa using hRel
  have h_eq := mk_eqr c (GM.h1 s * GM.h1 s-1) (GM.h1 1) hRel'
  calc
    φ (GM.h1 s-1) = 1 * φ (GM.h1 s-1) := by simp
    _ = ((φ (GM.h1 s))-1 * φ (GM.h1 s)) * φ (GM.h1 s-1) := by simp
    _ = (φ (GM.h1 s))-1 * (φ (GM.h1 s) * φ (GM.h1 s-1)) := by group
    _ = (φ (GM.h1 s))-1 * φ (GM.h1 s * GM.h1 s-1) := by rw [hφ_mul]
    _ = (φ (GM.h1 s))-1 * φ (GM.h1 1) := by rw [h_eq]
    _ = (φ (GM.h1 s))-1 * 1 := by rw [h1_one]
    _ = (φ (GM.h1 s))-1 := by simp

theorem h2_inv_mk (c : ℕ → ℕ) (s : ℂ×) :
  PresentedGroup.mk (GM.Rels c) (GM.h2 s-1) = (PresentedGroup.mk (GM.Rels c) (GM.h2 s))-1 := by
  rw [h2_eq_conj c s-1, h2_eq_conj c s]
  simp
  rw [h1_inv_mk c s]

```

group

```

theorem torusGens_conj (c : ℕ → ℕ) :
  ∀ t ∈ GM.torusGens c, ∀ g ∈ GM.unipGens c, t * g * t-1 ∈ GM.unipGens c := by
  intro t ht g hg
  rcases ht with ⟨s, hs⟩
  rcases hs with (hs | hs)
  ·
    rw [hs]
    exact h1_conj_unip c s g hg
  ·
    rw [hs]
    exact h2_conj_unip c s g hg

theorem torusGens_inv (c : ℕ → ℕ) :
  ∀ t ∈ GM.torusGens c, t-1 ∈ GM.torusGens c := by
  intro t ht
  rcases ht with ⟨s, hs | hs⟩
  · rw [hs]
    refine ⟨s-1, Or.inl ?_⟩
    rw [h1_inv_mk c s]
  · rw [hs]
    refine ⟨s-1, Or.inr ?_⟩
    rw [h2_inv_mk c s]

theorem conj_closure_of_gens {G : Type*} [Group G] (T : Set G) (x : G)
  (hx : ∀ g ∈ T, x * g * x-1 ∈ Subgroup.closure T) :
  ∀ u ∈ Subgroup.closure T, x * u * x-1 ∈ Subgroup.closure T := by
  intro u hu
  induction hu using Subgroup.closure_induction with
  | mem g hg =>
    exact hx g hg
  | one =>
    simp
  | mul a b ha hb iha ihb =>
    have h : x * (a * b) * x-1 = (x * a * x-1) * (x * b * x-1) := by group
    rw [h]
    exact Subgroup.mul_mem _ iha ihb
  | inv a ha iha =>
    have hxa : x * a-1 * x-1 = (x * a * x-1)-1 := by group
    rw [hxa]
    exact Subgroup.inv_mem _ iha

theorem closure_normalizes {G : Type*} [Group G] (S T : Set G)
  (hinv : ∀ t ∈ S, t-1 ∈ S)
  (hconj : ∀ t ∈ S, ∀ u ∈ Subgroup.closure T, t * u * t-1 ∈ Subgroup.closure T) :
  ∀ h ∈ Subgroup.closure S, ∀ u ∈ Subgroup.closure T, h * u * h-1 ∈ Subgroup.closure T := by
  intro h hh
  induction hh using Subgroup.closure_induction_left with
  | one =>
    intro u hu
    simp using hu
  | mul_left x hx y hy ih =>
    intro u hu
    have hcalc : (x * y) * u * (x * y)-1 = x * (y * u * y-1) * x-1 := by
      simp [mul_assoc]
    rw [hcalc]
    exact hconj x hx (y * u * y-1) (ih u hu)
  | inv_mul_cancel x hx y hy ih =>

```

```

intro u hu
have hcalc :  $(x^{-1} * y) * u * (x^{-1} * y)^{-1} = x^{-1} * (y * u * y^{-1}) * x$  := by
  simp [mul_assoc]
rw [hcalc]
have hcalc2 :  $x^{-1} * (y * u * y^{-1}) * x = x^{-1} * (y * u * y^{-1}) * (x^{-1})^{-1}$  := by
  simp
rw [hcalc2]
have hx_inv :  $x^{-1} \in S$  := hinv x hx
exact hconj  $x^{-1}$  hx_inv (y * u * y-1) (ih u hu)

theorem solution (c :  $\mathbb{N} \rightarrow \mathbb{N}$ ) :
   $\forall h \in \text{GM.torusSub } c, \forall u \in \text{GM.unipSub } c, h * u * h^{-1} \in \text{GM.unipSub } c$  := by
intro h hh u hu
exact closure_normalizes (GM.torusGens c) (GM.unipGens c) (torusGens_inv c)
(fun t ht => conj_closure_of_gens (GM.unipGens c) t
  (fun g hg => Subgroup.subset_closure (torusGens_conj c t ht g hg))) h hh u hu

```